```python
import pandas as pd

df=pd.read_csv(r"C:\Users\ishus\Downloads\Indian-Resturants (2).csv")
print(df.head())
```

```
    res_id                             name      establishment  \
0  3400299                       Bikanervala    ['Quick Bites']
1  3400005  Mama Chicken Mama Franky House    ['Quick Bites']
2  3401013                     Bhagat Halwai    ['Quick Bites']
3  3400290                     Bhagat Halwai    ['Quick Bites']
4  3401744     The Salt Cafe Kitchen & Bar  ['Casual Dining']

                                                  url  \
0  https://www.zomato.com/agra/bikanervala-khanda...
1  https://www.zomato.com/agra/mama-chicken-mama-...
2  https://www.zomato.com/agra/bhagat-halwai-2-sh...
3  https://www.zomato.com/agra/bhagat-halwai-civi...
4  https://www.zomato.com/agra/the-salt-cafe-kitc...

                                            address  city  city_id  \
0  Kalyani Point, Near Tulsi Cinema, Bypass Road,...  Agra       34
1        Main Market, Sadar Bazaar, Agra Cantt, Agra  Agra       34
2  62/1, Near Easy Day, West Shivaji Nagar, Goalp...  Agra       34
3  Near Anjana Cinema, Nehru Nagar, Civil Lines, ...  Agra       34
4        1C,3rd Floor, Fatehabad Road, Tajganj, Agra  Agra       34

      locality   latitude  longitude  ... price_range  currency  \
0     Khandari  27.211450  78.002381  ...           2       Rs.
1   Agra Cantt  27.160569  78.011583  ...           2       Rs.
2     Shahganj  27.182938  77.979684  ...           1       Rs.
3  Civil Lines  27.205668  78.004799  ...           1       Rs.
4      Tajganj  27.157709  78.052421  ...           3       Rs.

                                          highlights aggregate_rating
\
0  ['Lunch', 'Takeaway Available', 'Credit Card',...              4.4

1  ['Delivery', 'No Alcohol Available', 'Dinner',...              4.4

2  ['No Alcohol Available', 'Dinner', 'Takeaway A...              4.2

3  ['Takeaway Available', 'Credit Card', 'Lunch',...              4.3

4  ['Lunch', 'Serves Alcohol', 'Cash', 'Credit Ca...              4.9


  rating_text  votes  photo_count opentable_support delivery  takeaway

0   Very Good    814          154               0.0       -1        -1

1   Very Good   1203          161               0.0       -1        -1
```

```
2   Very Good    801          107               0.0        1        -1

3   Very Good    693          157               0.0        1        -1

4   Excellent    470          291               0.0        1        -1


[5 rows x 26 columns]
```

```python
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

df.shape
df.info()
df.duplicated().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211944 entries, 0 to 211943
Data columns (total 26 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   res_id                211944 non-null  int64
 1   name                  211944 non-null  object
 2   establishment         211944 non-null  object
 3   url                   211944 non-null  object
 4   address               211810 non-null  object
 5   city                  211944 non-null  object
 6   city_id               211944 non-null  int64
 7   locality              211944 non-null  object
 8   latitude              211944 non-null  float64
 9   longitude             211944 non-null  float64
 10  zipcode               48757 non-null   object
 11  country_id            211944 non-null  int64
 12  locality_verbose      211944 non-null  object
 13  cuisines              210553 non-null  object
 14  timings               208070 non-null  object
 15  average_cost_for_two  211944 non-null  int64
 16  price_range           211944 non-null  int64
 17  currency              211944 non-null  object
 18  highlights            211944 non-null  object
 19  aggregate_rating      211944 non-null  float64
 20  rating_text           211944 non-null  object
 21  votes                 211944 non-null  int64
 22  photo_count           211944 non-null  int64
 23  opentable_support     211896 non-null  float64
 24  delivery              211944 non-null  int64
 25  takeaway              211944 non-null  int64
dtypes: float64(4), int64(9), object(13)
memory usage: 42.0+ MB
```

```
np.int64(151527)
```

```
df.columns
```

```
Index(['res_id', 'name', 'establishment', 'url', 'address', 'city',
'city_id',
       'locality', 'latitude', 'longitude', 'zipcode', 'country_id',
       'locality_verbose', 'cuisines', 'timings',
'average_cost_for_two',
       'price_range', 'currency', 'highlights', 'aggregate_rating',
       'rating_text', 'votes', 'photo_count', 'opentable_support',
'delivery',
       'takeaway'],
      dtype='object')
```

```
df.describe().T
```

```
                          count          mean           std   min  \
res_id                 211944.0  1.349411e+07  7.883722e+06  50.0
city_id                211944.0  4.746785e+03  5.568766e+03   1.0
latitude               211944.0  2.149976e+01  2.278133e+01   0.0
longitude              211944.0  7.761528e+01  7.500104e+00   0.0
country_id             211944.0  1.000000e+00  0.000000e+00   1.0
average_cost_for_two   211944.0  5.958122e+02  6.062394e+02   0.0
price_range            211944.0  1.882535e+00  8.929891e-01   1.0
aggregate_rating       211944.0  3.395937e+00  1.283642e+00   0.0
votes                  211944.0  3.780019e+02  9.253334e+02 -18.0
photo_count            211944.0  2.569712e+02  8.676689e+02   0.0
opentable_support      211896.0  0.000000e+00  0.000000e+00   0.0
delivery               211944.0 -2.559072e-01  9.641721e-01  -1.0
takeaway               211944.0 -1.000000e+00  0.000000e+00  -1.0

                                25%           50%           75%
max
res_id                 3.301027e+06  1.869573e+07  1.881297e+07
1.915979e+07
city_id                1.100000e+01  3.400000e+01  1.130600e+04
1.135400e+04
latitude               1.549607e+01  2.251449e+01  2.684167e+01
1.000000e+04
longitude              7.487796e+01  7.742597e+01  8.021932e+01
9.183277e+01
country_id             1.000000e+00  1.000000e+00  1.000000e+00
1.000000e+00
average_cost_for_two   2.500000e+02  4.000000e+02  7.000000e+02
3.000000e+04
price_range            1.000000e+00  2.000000e+00  2.000000e+00
4.000000e+00
aggregate_rating       3.300000e+00  3.800000e+00  4.100000e+00
4.900000e+00
```

```
votes                  1.600000e+01  1.000000e+02  3.620000e+02
4.253900e+04
photo_count            3.000000e+00  1.800000e+01  1.280000e+02
1.770200e+04
opentable_support      0.000000e+00  0.000000e+00  0.000000e+00
0.000000e+00
delivery              -1.000000e+00 -1.000000e+00  1.000000e+00
1.000000e+00
takeaway              -1.000000e+00 -1.000000e+00 -1.000000e+00 -
1.000000e+00
```

```
df.isnull().sum()
```

```
res_id                       0
name                         0
establishment                0
url                          0
address                    134
city                         0
city_id                      0
locality                     0
latitude                     0
longitude                    0
zipcode                 163187
country_id                   0
locality_verbose             0
cuisines                  1391
timings                   3874
average_cost_for_two         0
price_range                  0
currency                     0
highlights                   0
aggregate_rating             0
rating_text                  0
votes                        0
photo_count                  0
opentable_support           48
delivery                     0
takeaway                     0
dtype: int64
```

The zipcode column had a large number of missing values and did not contribute significantly to the analysis of restaurant success factors. Therefore, it was removed to reduce noise and complexity.

```
df.drop(columns=['zipcode'], inplace=True)

df
```

```
          res_id                              name      establishment  \
0        3400299                       Bikanervala    ['Quick Bites']
1        3400005  Mama Chicken Mama Franky House    ['Quick Bites']
2        3401013                    Bhagat Halwai    ['Quick Bites']
3        3400290                    Bhagat Halwai    ['Quick Bites']
4        3401744     The Salt Cafe Kitchen & Bar  ['Casual Dining']
...          ...                              ...                ...
211939   3202251  Kali Mirch Cafe And Restaurant  ['Casual Dining']
211940   3200996                      Raju Omlet    ['Quick Bites']
211941  18984164                 The Grand Thakar  ['Casual Dining']
211942   3201138                          Subway    ['Quick Bites']
211943  18879846     Freshco's - The Health Cafe           ['Café']

                                                     url  \
0       https://www.zomato.com/agra/bikanervala-khanda...
1       https://www.zomato.com/agra/mama-chicken-mama-...
2       https://www.zomato.com/agra/bhagat-halwai-2-sh...
3       https://www.zomato.com/agra/bhagat-halwai-civi...
4       https://www.zomato.com/agra/the-salt-cafe-kitc...
...                                                   ...
211939  https://www.zomato.com/vadodara/kali-mirch-caf...
211940  https://www.zomato.com/vadodara/raju-omlet-kar...
211941  https://www.zomato.com/vadodara/the-grand-thak...
211942  https://www.zomato.com/vadodara/subway-1-akota...
211943  https://www.zomato.com/vadodara/freshcos-the-h...

                                                 address      city
city_id  \
0       Kalyani Point, Near Tulsi Cinema, Bypass Road,...      Agra
34
1             Main Market, Sadar Bazaar, Agra Cantt, Agra      Agra
34
2       62/1, Near Easy Day, West Shivaji Nagar, Goalp...      Agra
34
3       Near Anjana Cinema, Nehru Nagar, Civil Lines, ...      Agra
34
4             1C,3rd Floor, Fatehabad Road, Tajganj, Agra      Agra
34
...                                                   ...       ...
...
211939  Manu Smriti Complex, Near Navrachna School, GI...  Vadodara
32
211940  Mahalaxmi Apartment, Opposite B O B, Karoli Ba...  Vadodara
32
211941  3rd Floor, Shreem Shalini Mall, Opposite Conqu...  Vadodara
32
211942  G-2, Vedant Platina, Near Cosmos, Akota, Vadodara  Vadodara
32
211943  Shop 7, Ground Floor, Opposite Natubhai Circle...  Vadodara
32
```

```
            locality   latitude  longitude  ...  price_range
currency  \
0           Khandari  27.211450  78.002381  ...            2      Rs.

1         Agra Cantt  27.160569  78.011583  ...            2      Rs.

2           Shahganj  27.182938  77.979684  ...            1      Rs.

3        Civil Lines  27.205668  78.004799  ...            1      Rs.

4            Tajganj  27.157709  78.052421  ...            3      Rs.

...              ...        ...        ...  ...          ...      ...

211939    Fatehgunj  22.336931  73.192356  ...            2      Rs.

211940   Karelibaug  22.322455  73.197203  ...            1      Rs.

211941     Alkapuri  22.310563  73.171163  ...            2      Rs.

211942        Akota  22.270027  73.143068  ...            2      Rs.

211943     Vadiwadi  22.309935  73.158768  ...            2      Rs.


                                                   highlights
aggregate_rating  \
0        ['Lunch', 'Takeaway Available', 'Credit Card',...
4.4
1        ['Delivery', 'No Alcohol Available', 'Dinner',...
4.4
2        ['No Alcohol Available', 'Dinner', 'Takeaway A...
4.2
3        ['Takeaway Available', 'Credit Card', 'Lunch',...
4.3
4        ['Lunch', 'Serves Alcohol', 'Cash', 'Credit Ca...
4.9
...                                                    ...
...
211939  ['Dinner', 'Cash', 'Lunch', 'Delivery', 'Indoo...
4.1
211940  ['Dinner', 'Cash', 'Takeaway Available', 'Debi...
4.1
211941  ['Dinner', 'Cash', 'Debit Card', 'Lunch', 'Tak...
4.0
211942  ['Dinner', 'Delivery', 'Credit Card', 'Lunch',...
3.7
211943  ['Dinner', 'Cash', 'Takeaway Available', 'Debi...
4.0
```

```
        rating_text  votes photo_count opentable_support  delivery
takeaway
0         Very Good    814         154               0.0        -1
-1
1         Very Good   1203         161               0.0        -1
-1
2         Very Good    801         107               0.0         1
-1
3         Very Good    693         157               0.0         1
-1
4         Excellent    470         291               0.0         1
-1
...             ...    ...         ...               ...       ...
...
211939    Very Good    243          40               0.0        -1
-1
211940    Very Good    187          40               0.0         1
-1
211941    Very Good    111          38               0.0        -1
-1
211942         Good    128          34               0.0         1
-1
211943    Very Good     93          53               0.0         1
-1

[211944 rows x 25 columns]
```

filling nan values or missing values

Cuisines

```
df['cuisines'].fillna(df['cuisines'].mode()[0], inplace=True)
df['cuisines'].isnull().sum()
```

```
np.int64(0)
```

Address

```
df['address'].fillna(df['address'].mode()[0],inplace=True)
df['address'].isnull().sum()
```

```
np.int64(0)
```

Timings

```
df['timings'].fillna(df['timings'].mode()[0],inplace=True)
df['timings'].isnull().sum()
```

```
np.int64(0)
```

Opentable_support

```
df['opentable_support'].fillna(df['opentable_support'].mean(),inplace=
True)
df['opentable_support'].isnull().sum()
```

```
np.int64(0)
```

```
df.drop_duplicates()
```

```
          res_id                              name        establishment
\
0        3400299                       Bikanervala      ['Quick Bites']

1        3400005  Mama Chicken Mama Franky House      ['Quick Bites']

2        3401013                     Bhagat Halwai      ['Quick Bites']

3        3400290                     Bhagat Halwai      ['Quick Bites']

4        3401744     The Salt Cafe Kitchen & Bar    ['Casual Dining']

...          ...                               ...                   ...

211882  19142822           Shree Janta Ice Cream  ['Dessert Parlour']

211925  18984164                 The Grand Thakar    ['Casual Dining']

211926  18019952                       Geeta lodge    ['Casual Dining']

211940   3200996                        Raju Omlet      ['Quick Bites']

211942   3201138                            Subway      ['Quick Bites']


                                                     url  \
0       https://www.zomato.com/agra/bikanervala-khanda...
1       https://www.zomato.com/agra/mama-chicken-mama-...
2       https://www.zomato.com/agra/bhagat-halwai-2-sh...
3       https://www.zomato.com/agra/bhagat-halwai-civi...
4       https://www.zomato.com/agra/the-salt-cafe-kitc...
...                                                   ...
211882  https://www.zomato.com/vadodara/shree-janta-ic...
211925  https://www.zomato.com/vadodara/the-grand-thak...
211926  https://www.zomato.com/vadodara/geeta-lodge-al...
211940  https://www.zomato.com/vadodara/raju-omlet-kar...
211942  https://www.zomato.com/vadodara/subway-1-akota...

                                                 address      city
city_id  \
0       Kalyani Point, Near Tulsi Cinema, Bypass Road,...      Agra
34
```

```
1             Main Market, Sadar Bazaar, Agra Cantt, Agra      Agra
34
2       62/1, Near Easy Day, West Shivaji Nagar, Goalp...      Agra
34
3       Near Anjana Cinema, Nehru Nagar, Civil Lines, ...      Agra
34
4             1C,3rd Floor, Fatehabad Road, Tajganj, Agra      Agra
34
...                                                     ...       ...
...
211882  Ground Floor, 5 Ronak Plaza, Tulsidham Char Ra...  Vadodara
32
211925  3rd Floor, Shreem Shalini Mall, Opposite Conqu...  Vadodara
32
211926  Shop 11, Ground Floor, Atlantis K-10, Tower A,...  Vadodara
32
211940  Mahalaxmi Apartment, Opposite B O B, Karoli Ba...  Vadodara
32
211942  G-2, Vedant Platina, Near Cosmos, Akota, Vadodara  Vadodara
32

           locality   latitude  longitude  ...  price_range
currency  \
0          Khandari  27.211450  78.002381  ...            2      Rs.

1        Agra Cantt  27.160569  78.011583  ...            2      Rs.

2          Shahganj  27.182938  77.979684  ...            1      Rs.

3       Civil Lines  27.205668  78.004799  ...            1      Rs.

4           Tajganj  27.157709  78.052421  ...            3      Rs.

...             ...        ...        ...  ...          ...      ...

211882    Manjalpur  22.270516  73.196408  ...            1      Rs.

211925     Alkapuri  22.310563  73.171163  ...            2      Rs.

211926     Alkapuri  22.317731  73.168107  ...            1      Rs.

211940   Karelibaug  22.322455  73.197203  ...            1      Rs.

211942        Akota  22.270027  73.143068  ...            2      Rs.


                                                   highlights
aggregate_rating  \
0       ['Lunch', 'Takeaway Available', 'Credit Card',...
4.4
```

```
1       ['Delivery', 'No Alcohol Available', 'Dinner',...
4.4
2       ['No Alcohol Available', 'Dinner', 'Takeaway A...
4.2
3       ['Takeaway Available', 'Credit Card', 'Lunch',...
4.3
4       ['Lunch', 'Serves Alcohol', 'Cash', 'Credit Ca...
4.9
...                                                   ...
...
211882  ['Cash', 'Takeaway Available', 'Delivery', 'In...
2.9
211925  ['Dinner', 'Cash', 'Debit Card', 'Lunch', 'Tak...
4.0
211926  ['Dinner', 'Cash', 'Credit Card', 'Lunch', 'Ta...
3.9
211940  ['Dinner', 'Cash', 'Takeaway Available', 'Debi...
4.1
211942  ['Dinner', 'Delivery', 'Credit Card', 'Lunch',...
3.7

        rating_text  votes photo_count opentable_support  delivery
takeaway
0         Very Good    814         154               0.0        -1
-1
1         Very Good   1203         161               0.0        -1
-1
2         Very Good    801         107               0.0         1
-1
3         Very Good    693         157               0.0         1
-1
4         Excellent    470         291               0.0         1
-1
...             ...    ...         ...               ...       ...
...
211882      Average      4           1               0.0         1
-1
211925    Very Good    111          38               0.0        -1
-1
211926         Good    207          14               0.0        -1
-1
211940    Very Good    187          40               0.0         1
-1
211942         Good    128          34               0.0         1
-1

[60411 rows x 25 columns]

befor=211944
after=60411
```

```
print('we have droped duplicate rows','before shape of data :',befor
      , 'after droping the rows : ',after)
```

```
we have droped duplicate rows before shape of data : 211944 after
droping the rows :  60411
```

```
df['name'].nunique()
```

```
41100
```

```
df['cuisines'].nunique()
```

```
9382
```

```
df['city'].nunique()
```

```
99
```

Univeriate Analysis

1 The histogram and KDE plot show that most restaurant ratings are concentrated between 3.2 and 4.5, with a peak around 4.0, indicating that most restaurants receive moderate to high ratings. The distribution is slightly skewed, suggesting ratings are more concentrated toward higher values, while very low ratings are less common.

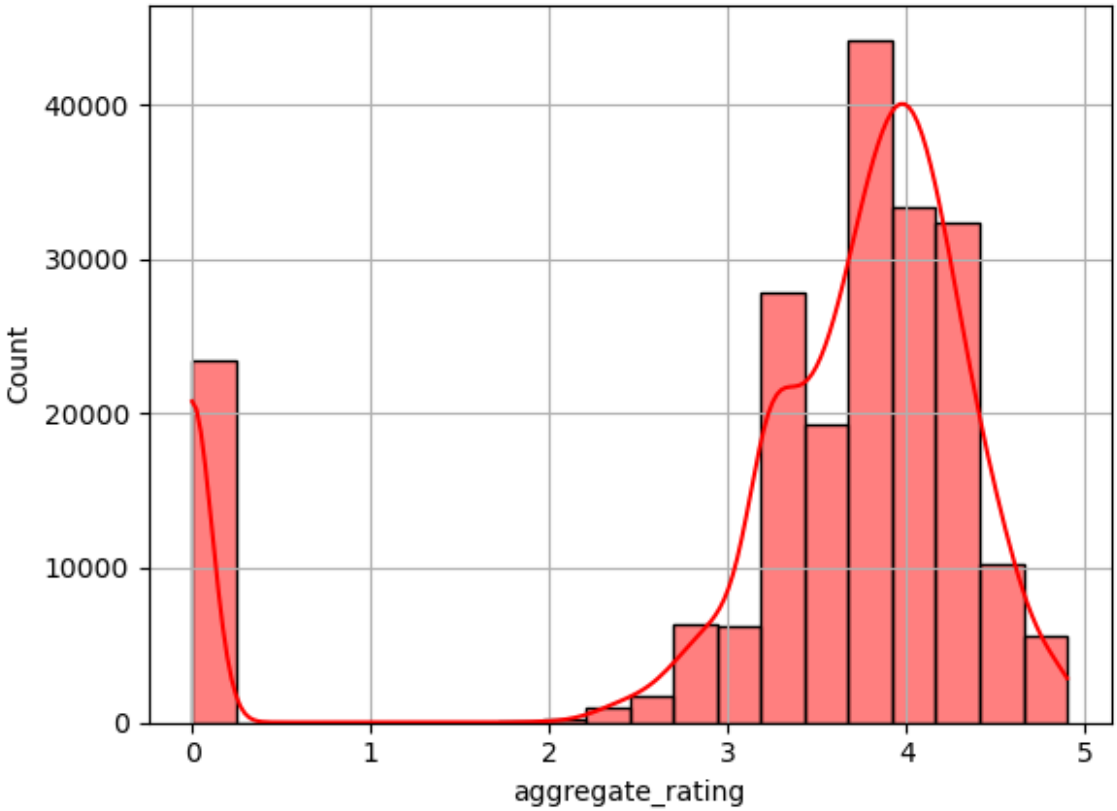
2 A few rating values near 0 (or extreme ends) may indicate anomalies or unrated restaurants

Rating

```
import seaborn as sns
import matplotlib.pyplot as plt
df['aggregate_rating'] = pd.to_numeric(df['aggregate_rating'],
errors='coerce')
print(df['aggregate_rating'].describe())
sns.histplot(df['aggregate_rating'],kde=True, bins=20,color='red')
plt.grid(True)
plt.show()
```

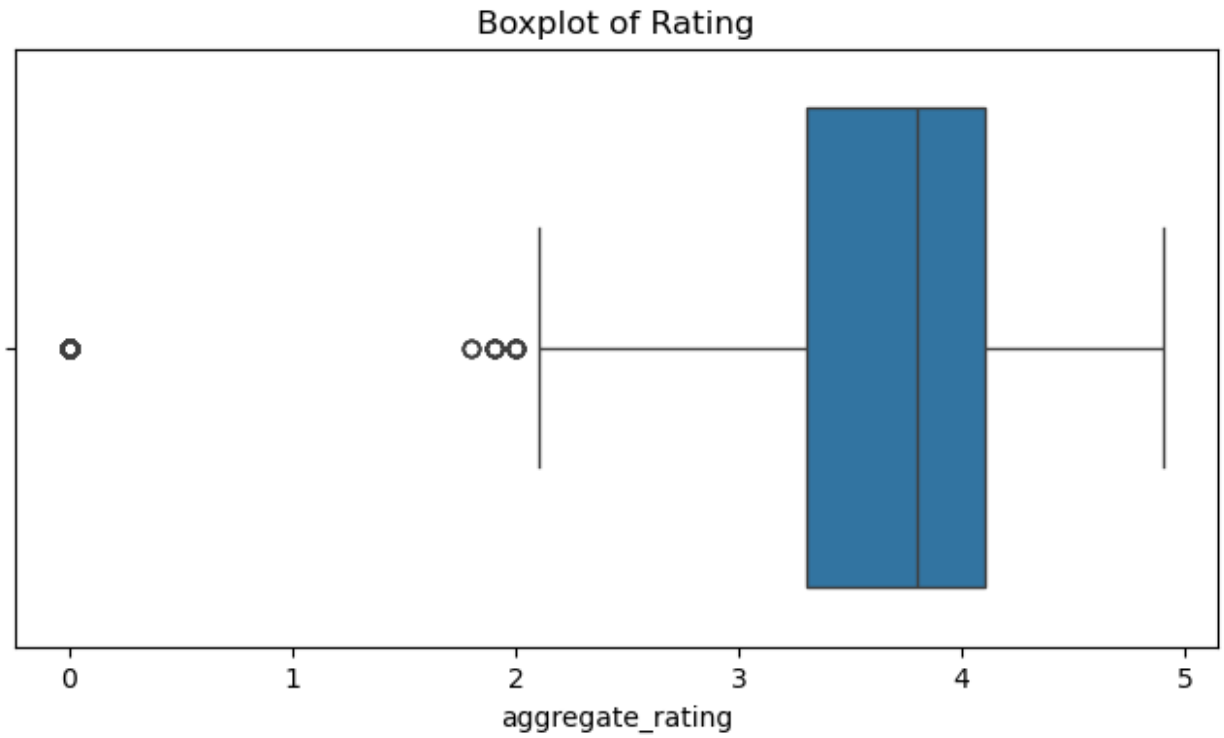
```
count    211944.000000
mean          3.395937
std           1.283642
min           0.000000
25%           3.300000
```

```
50%            3.800000
75%            4.100000
max            4.900000
Name: aggregate_rating, dtype: float64
```



Box plot

```
plt.figure(figsize=(8,4))
sns.boxplot(x=df['aggregate_rating'])
plt.title("Boxplot of Rating")
plt.show()
```




```
df['aggregate_rating'].isnull().sum()

np.int64(0)

print(df.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211944 entries, 0 to 211943
Data columns (total 25 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   res_id                211944 non-null  int64
 1   name                  211944 non-null  object
 2   establishment         211944 non-null  object
 3   url                   211944 non-null  object
 4   address               211944 non-null  object
 5   city                  211944 non-null  object
 6   city_id               211944 non-null  int64
 7   locality              211944 non-null  object
 8   latitude              211944 non-null  float64
 9   longitude             211944 non-null  float64
 10  country_id            211944 non-null  int64
 11  locality_verbose      211944 non-null  object
 12  cuisines              211944 non-null  object
 13  timings               211944 non-null  object
 14  average_cost_for_two  211944 non-null  int64
 15  price_range           211944 non-null  int64
```

```
 16  currency              211944 non-null  object
 17  highlights            211944 non-null  object
 18  aggregate_rating      211944 non-null  float64
 19  rating_text           211944 non-null  object
 20  votes                 211944 non-null  int64
 21  photo_count           211944 non-null  int64
 22  opentable_support     211944 non-null  float64
 23  delivery              211944 non-null  int64
 24  takeaway              211944 non-null  int64
dtypes: float64(4), int64(9), object(12)
memory usage: 40.4+ MB
None
```

According to this boxplot all the values of price range column are very low which says the data is positive skewed while a high price value exist as outliers

```
df[['aggregate_rating','latitude','longitude']].corr()
```

```
                   aggregate_rating  latitude  longitude
aggregate_rating           1.000000 -0.002170   0.069011
latitude                  -0.002170  1.000000   0.047101
longitude                  0.069011  0.047101   1.000000
```

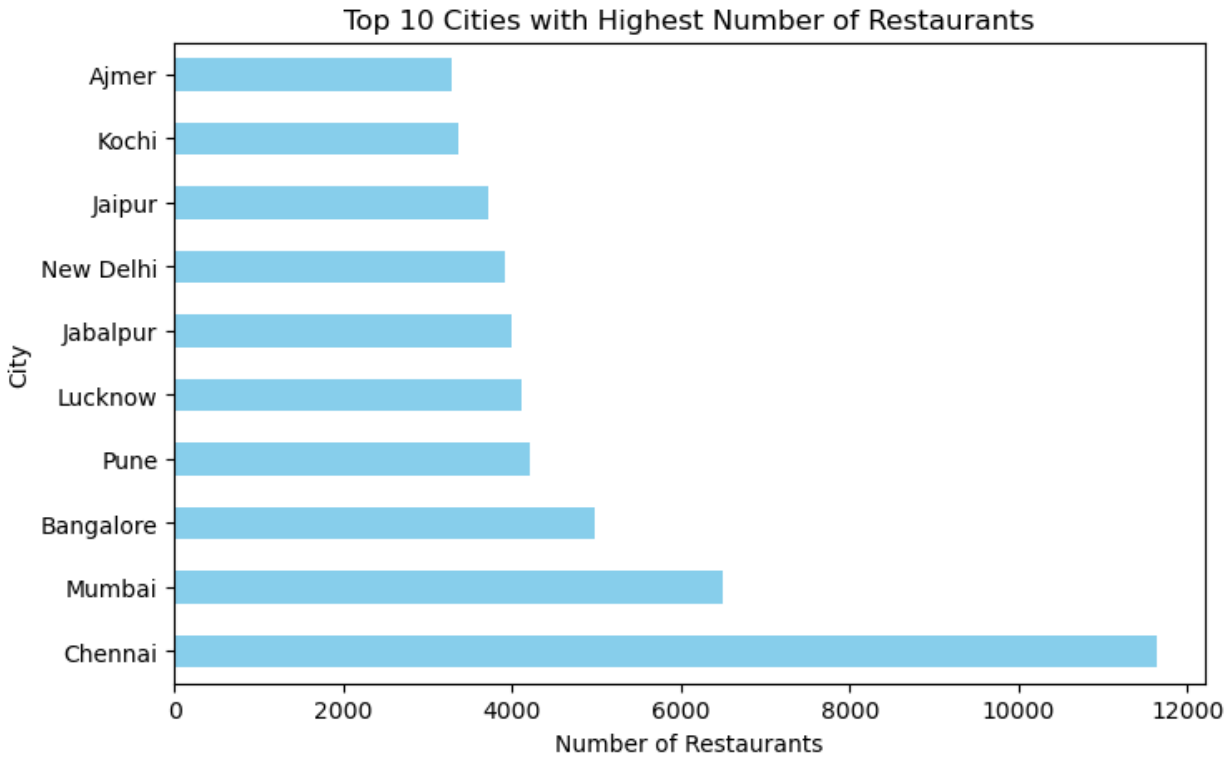
Location Analysis

We are plotting a horizontal bar plot to check which city have more number of resturant

This will give average rating by city and then sort it in ascending order and after that plot it in the form of horzontal bar graph

```
# City counts
city_counts = df['city'].value_counts()
print("Top 10 Cities with Most Restaurants:")
print(city_counts.head(10))
# City with highest number of restaurants
top_city = city_counts.idxmax()
count_top_city = city_counts.max()
print("\nCity with highest concentration of restaurants:", top_city)
print("Number of restaurants:", count_top_city)
plt.figure(figsize=(8,5))
city_counts.head(10).plot(kind='barh', color='skyblue')
plt.xlabel("Number of Restaurants")
plt.ylabel("City")
plt.title("Top 10 Cities with Highest Number of Restaurants")
plt.show()
```

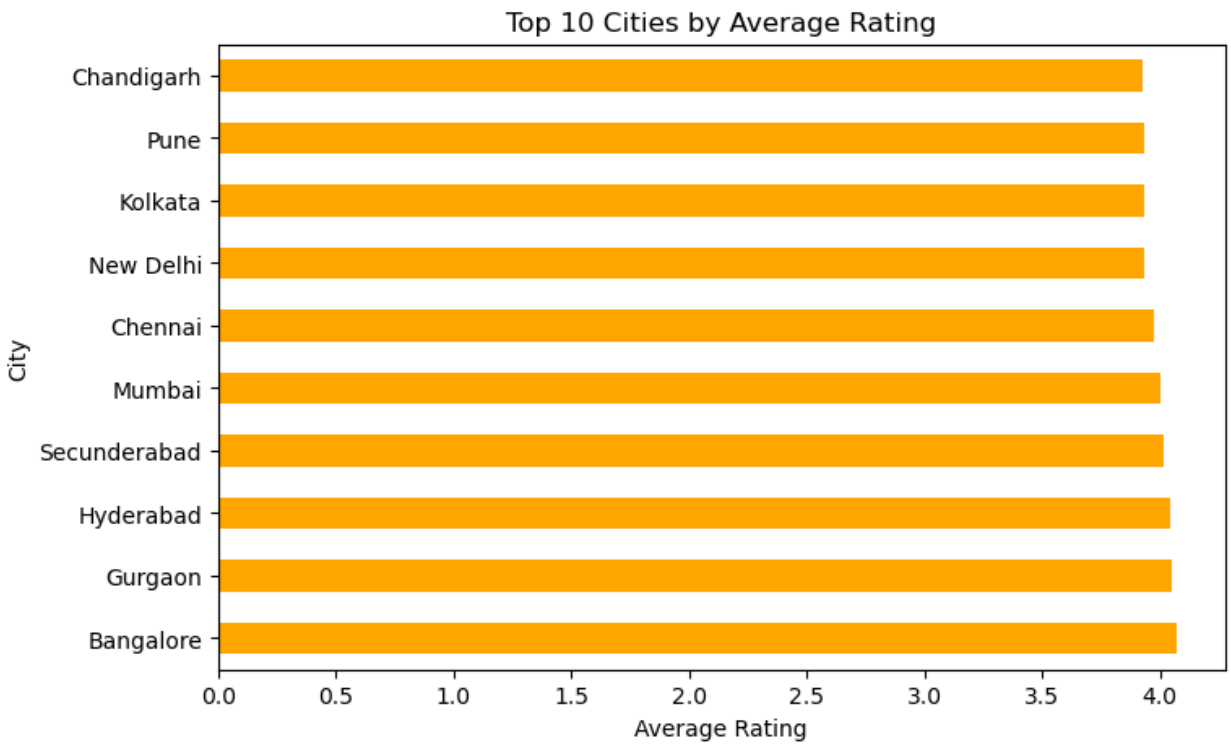
```
Top 10 Cities with Most Restaurants:
city
Chennai      11630
Mumbai        6497
Bangalore     4971
Pune          4217
Lucknow       4121
Jabalpur      3994
New Delhi     3918
Jaipur        3713
Kochi         3370
Ajmer         3277
Name: count, dtype: int64

City with highest concentration of restaurants: Chennai
Number of restaurants: 11630
```




```
city_rating = df.groupby('city')
['aggregate_rating'].mean().sort_values(ascending=False)
print("Top Cities by Average Rating:")
print(city_rating.head(10))
plt.figure(figsize=(8,5))
city_rating.head(10).plot(kind='barh', color='orange')
plt.xlabel("Average Rating")
plt.ylabel("City")
```

```
plt.title("Top 10 Cities by Average Rating")
plt.show()
```

```
Top Cities by Average Rating:
city
Bangalore       4.073567
Gurgaon         4.048837
Hyderabad       4.042747
Secunderabad    4.018579
Mumbai          4.004848
Chennai         3.973938
New Delhi       3.935988
Kolkata         3.935536
Pune            3.931705
Chandigarh      3.927081
Name: aggregate_rating, dtype: float64
```

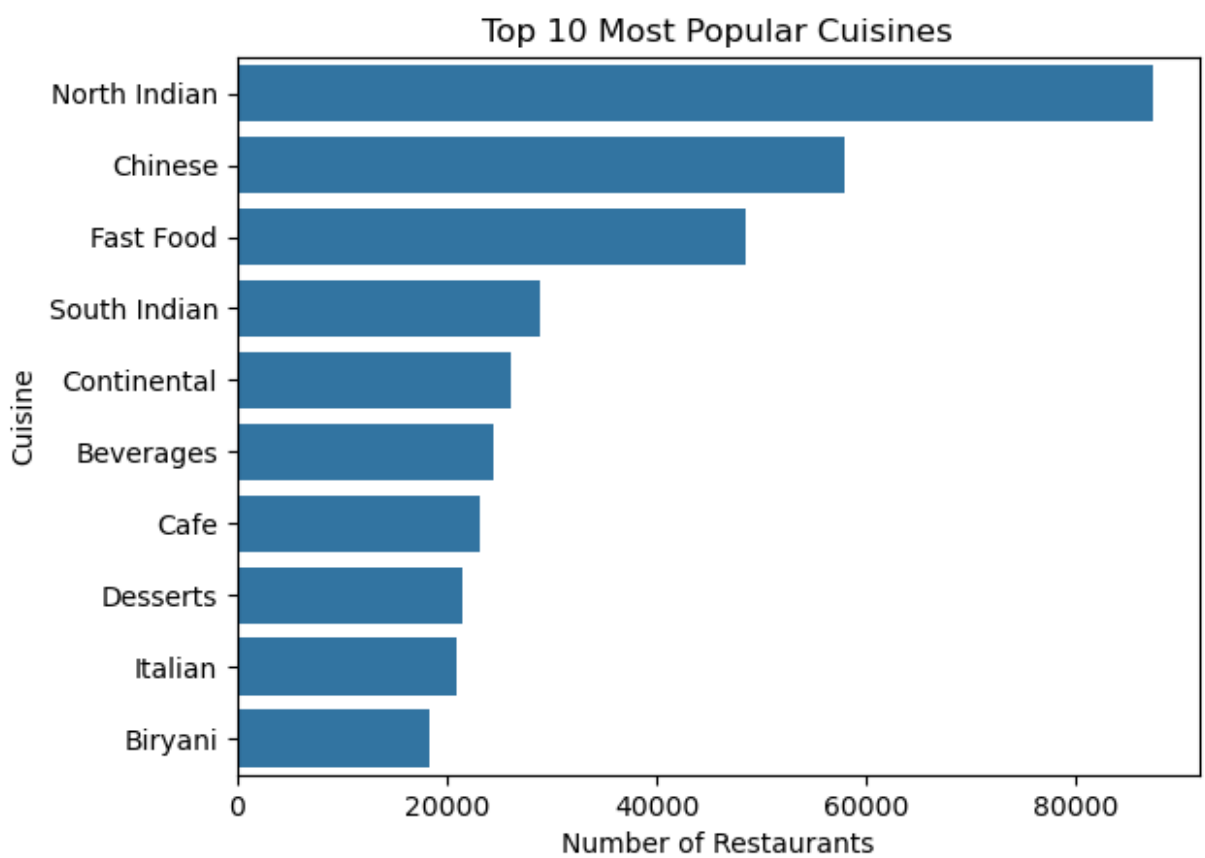


Cuisine Analysis:

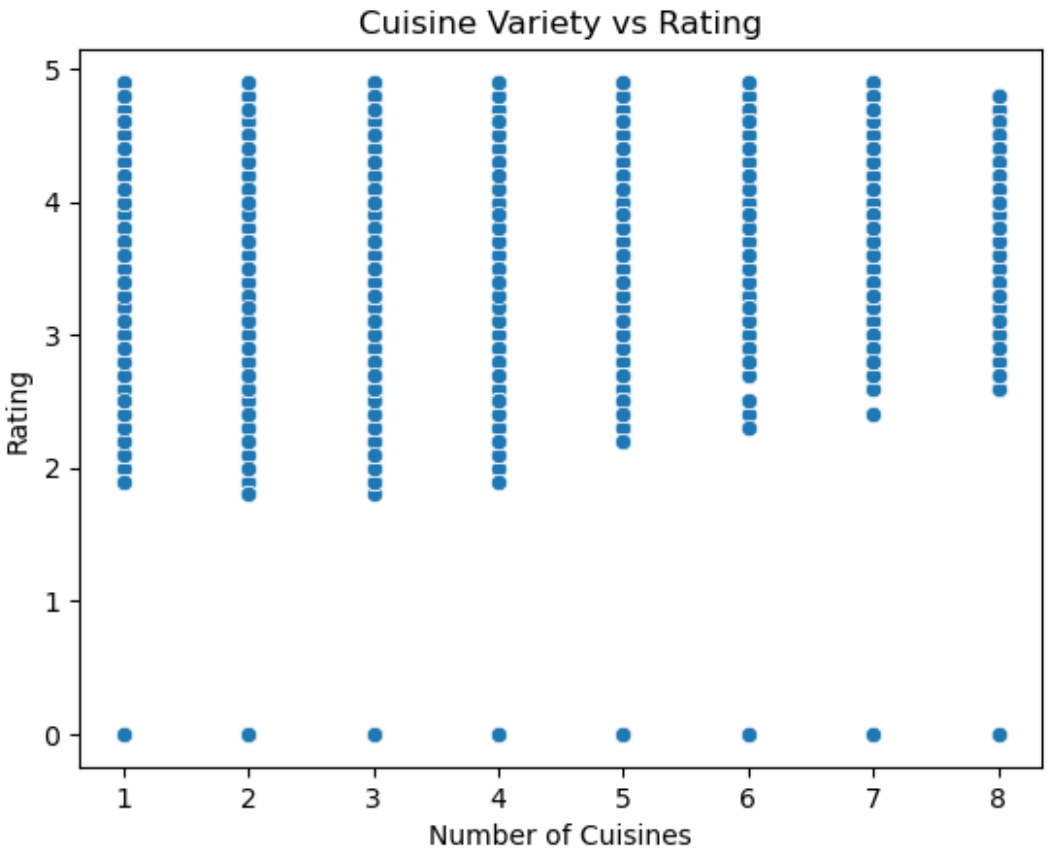
We are visualizing the cuisines column which tell us 10 most popular cuisine with respect to resturant

we are plotting number of cusines vs Rating

```
cuisine_split = df['cuisines'].str.split(',')
all_cuisines = []
for row in cuisine_split:
    for c in row:
        all_cuisines.append(c.strip())

import pandas as pd
cuisine_counts = pd.Series(all_cuisines).value_counts()
sns.barplot( x = cuisine_counts.head(10).values,y =
cuisine_counts.head(10).index)
plt.xlabel("Number of Restaurants")
plt.ylabel("Cuisine")
plt.title("Top 10 Most Popular Cuisines")
plt.show()
```



```
df['cuisine_count'] = df['cuisines'].str.split(',').str.len()
df.groupby('cuisine_count')['aggregate_rating'].mean()
sns.scatterplot(x = 'cuisine_count',y = 'aggregate_rating',data = df)
plt.xlabel("Number of Cuisines")
plt.ylabel("Rating")
plt.title("Cuisine Variety vs Rating")
plt.show()
```



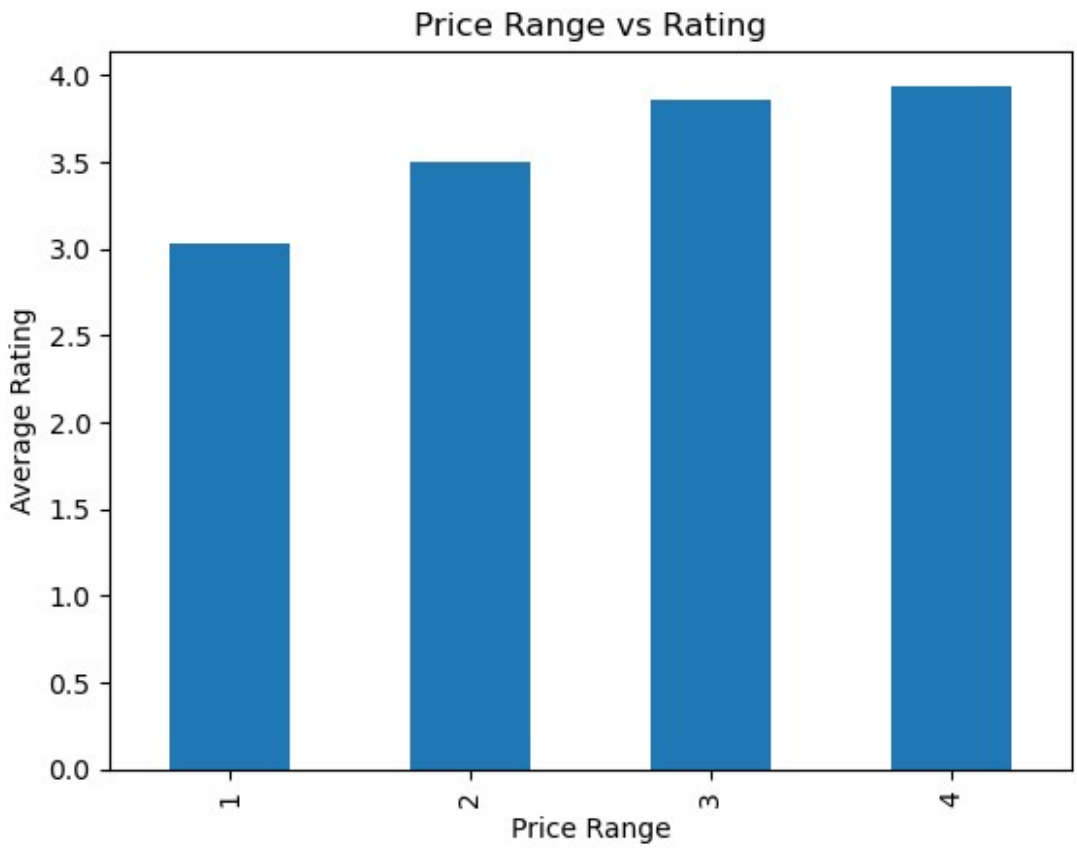
Price Range and Rating:

We are analysing the price range with respect to rating

same for price range for cost of 2 person in a restrant

```
df.groupby('price_range')['aggregate_rating'].mean()
avg_price_rating = df.groupby('price_range')
['aggregate_rating'].mean()

avg_price_rating.plot(kind='bar')
plt.xlabel("Price Range")
plt.ylabel("Average Rating")
```

```
plt.title("Price Range vs Rating")
plt.show()
```



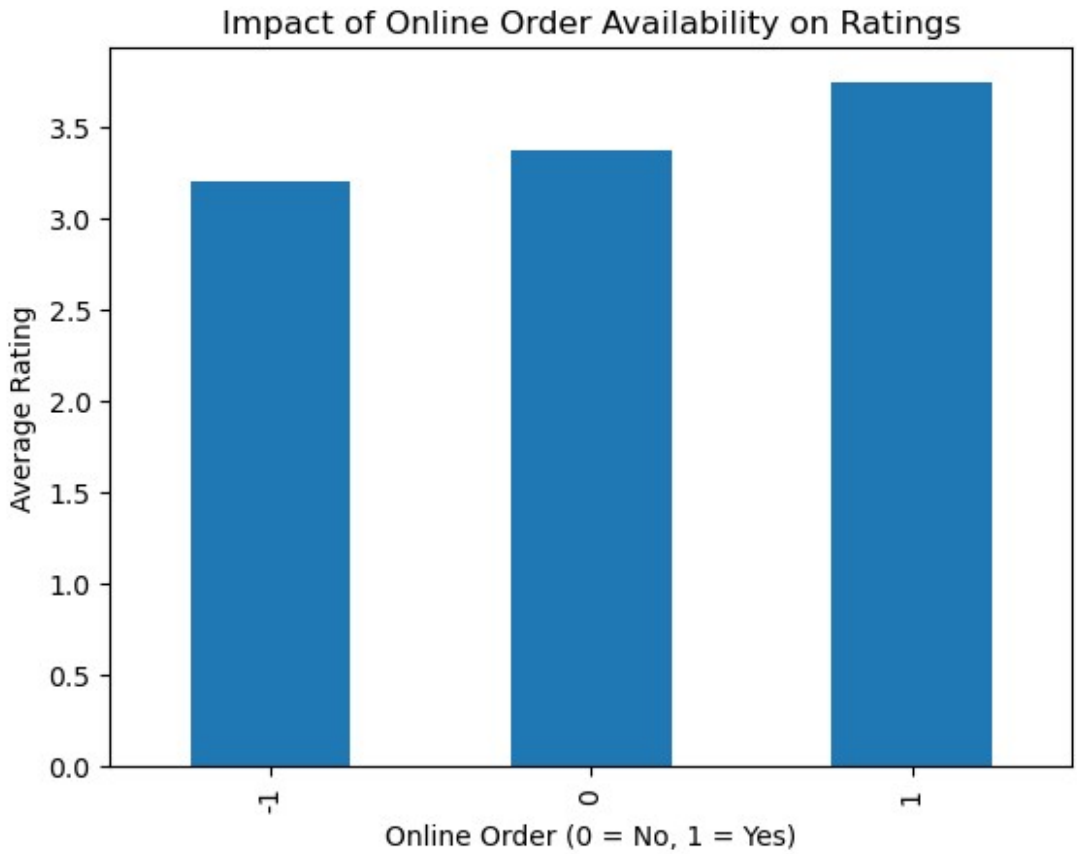

```
df.groupby('price_range')['average_cost_for_two'].mean()
avg_cost = df.groupby('price_range')['average_cost_for_two'].mean()
avg_cost.plot(kind='bar')
plt.xlabel("Price Range")
plt.ylabel("Average Cost for Two")
plt.title("Average Cost for Two in Each Price Category")
plt.show()
```

Online Order and Table Booking:

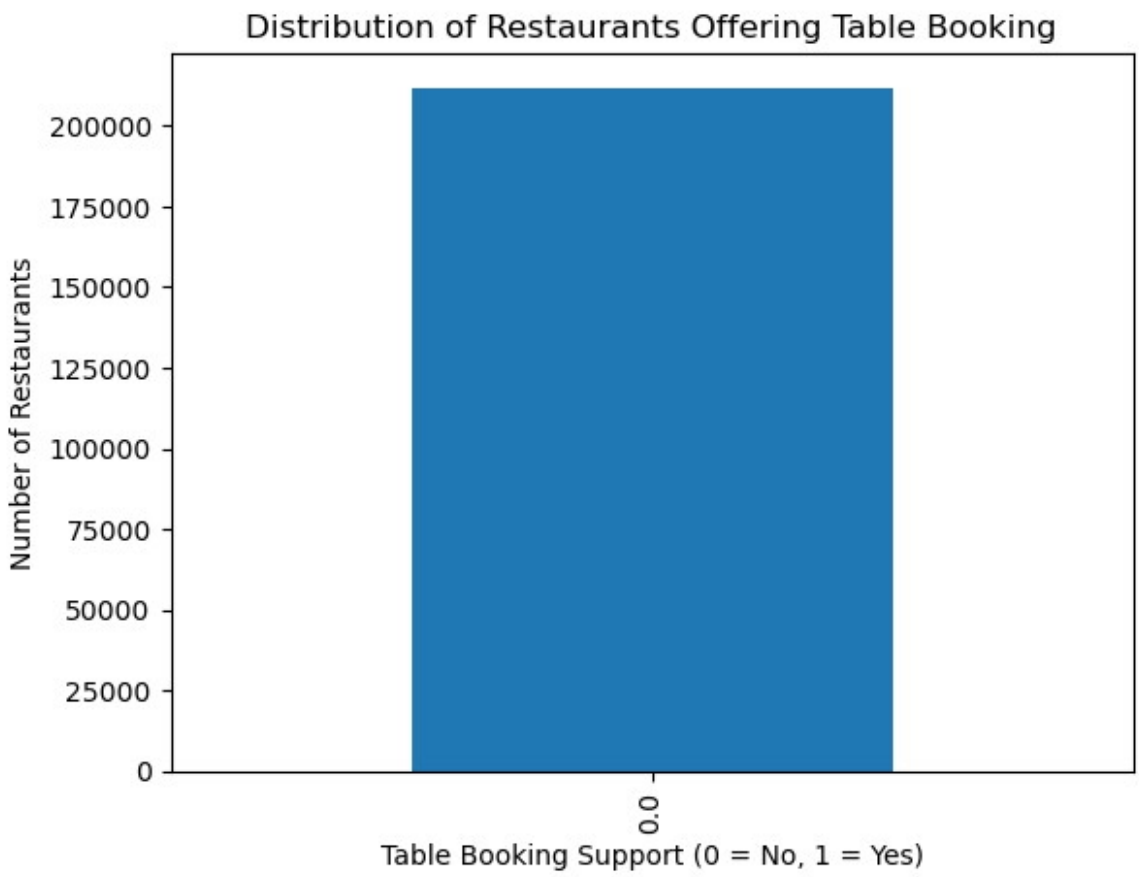
In the first plot we are analysing the relationship in between delivery and rating

In second plot we are just plotting if the the resturant offer open table support

```
df.groupby('delivery')['aggregate_rating'].mean()
avg_online = df.groupby('delivery')['aggregate_rating'].mean()
avg_online.plot(kind='bar')
plt.xlabel("Online Order (0 = No, 1 = Yes)")
plt.ylabel("Average Rating")
plt.title("Impact of Online Order Availability on Ratings")
plt.show()
```




```
df['opentable_support'].value_counts()
df['opentable_support'].value_counts().plot(kind='bar')
plt.xlabel("Table Booking Support (0 = No, 1 = Yes)")
plt.ylabel("Number of Restaurants")
plt.title("Distribution of Restaurants Offering Table Booking")
plt.show()
```



Top Restaurant Chains:

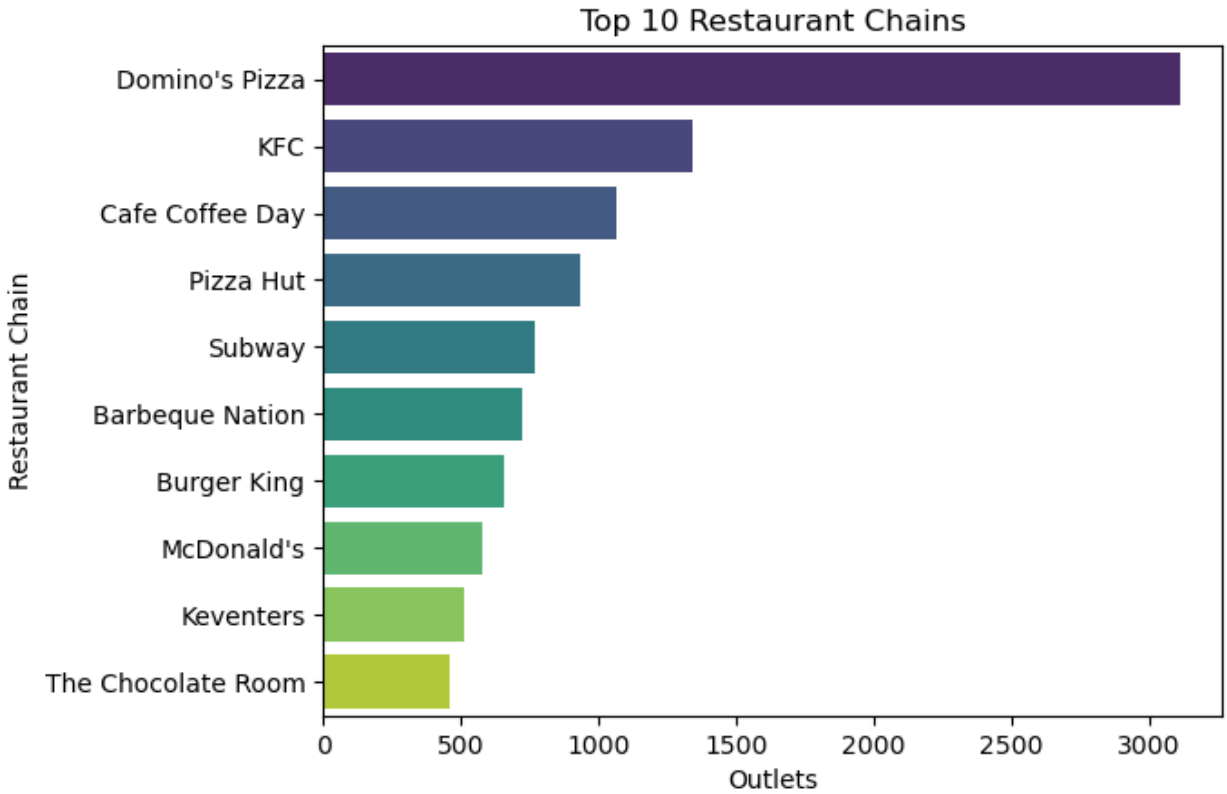
In First plot of horizontal bar chart for findinig the top 10 resturant chains

and a pie chart for analysing which resturant cover how much percentage in market

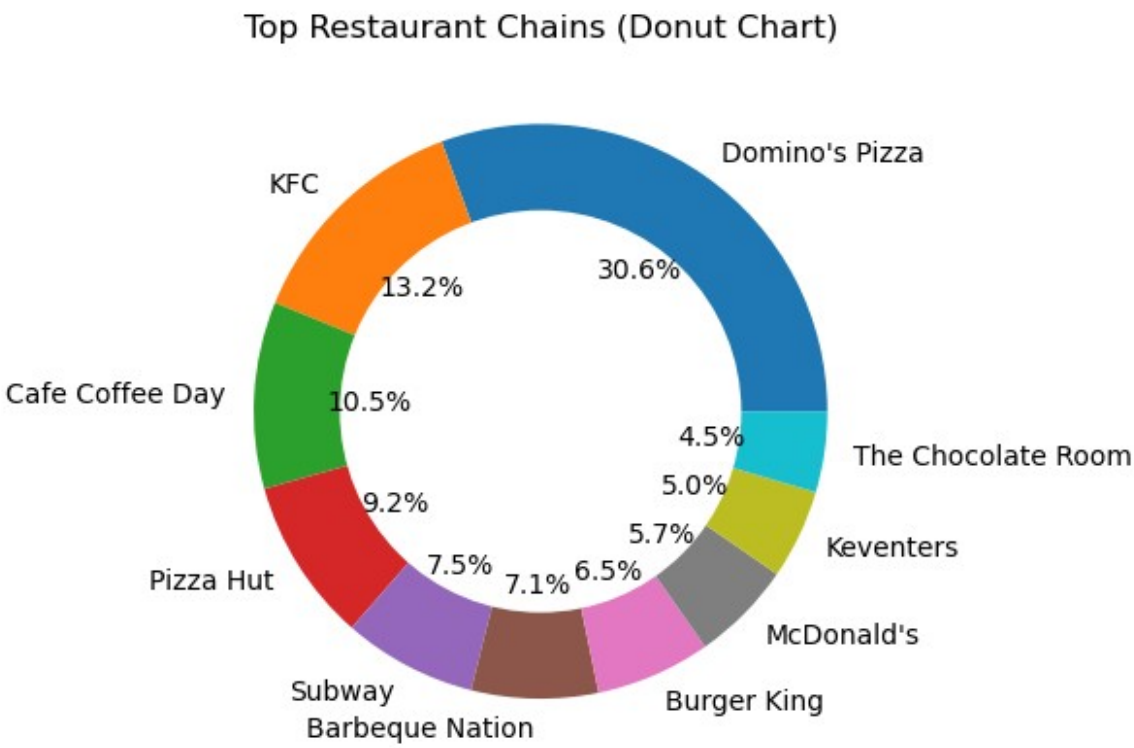
```
chain_counts = df['name'].value_counts().head(10)
print(chain_counts)
sns.barplot(x=chain_counts.values, y=chain_counts.index,
palette='viridis')
plt.xlabel("Outlets")
plt.ylabel("Restaurant Chain")
plt.title("Top 10 Restaurant Chains")
plt.show()
```

```
name
Domino's Pizza        3108
KFC                   1343
Cafe Coffee Day       1068
Pizza Hut              936
```

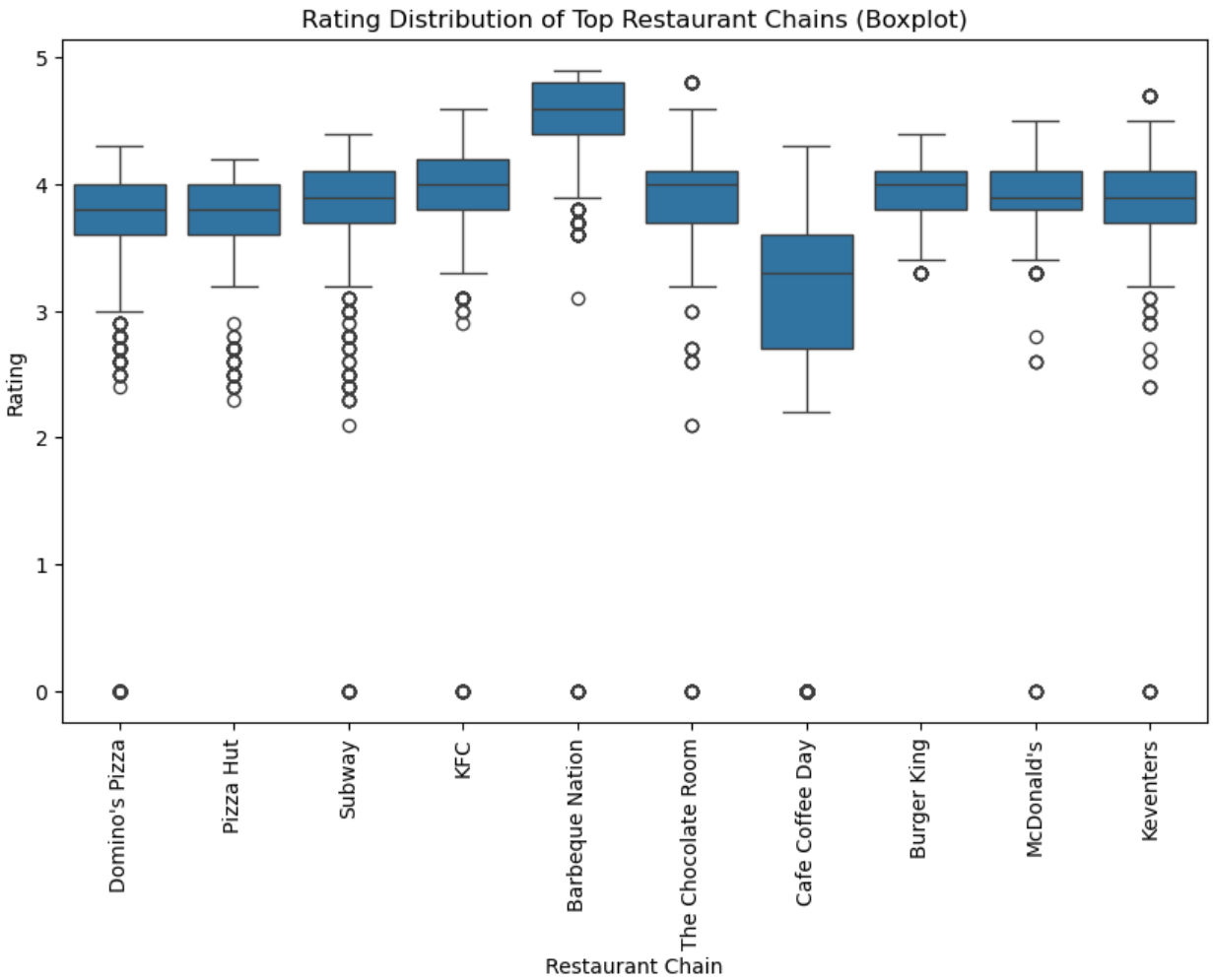
```
Subway                  766
Barbeque Nation         725
Burger King             658
McDonald's              578
Keventers               512
The Chocolate Room      461
Name: count, dtype: int64
```



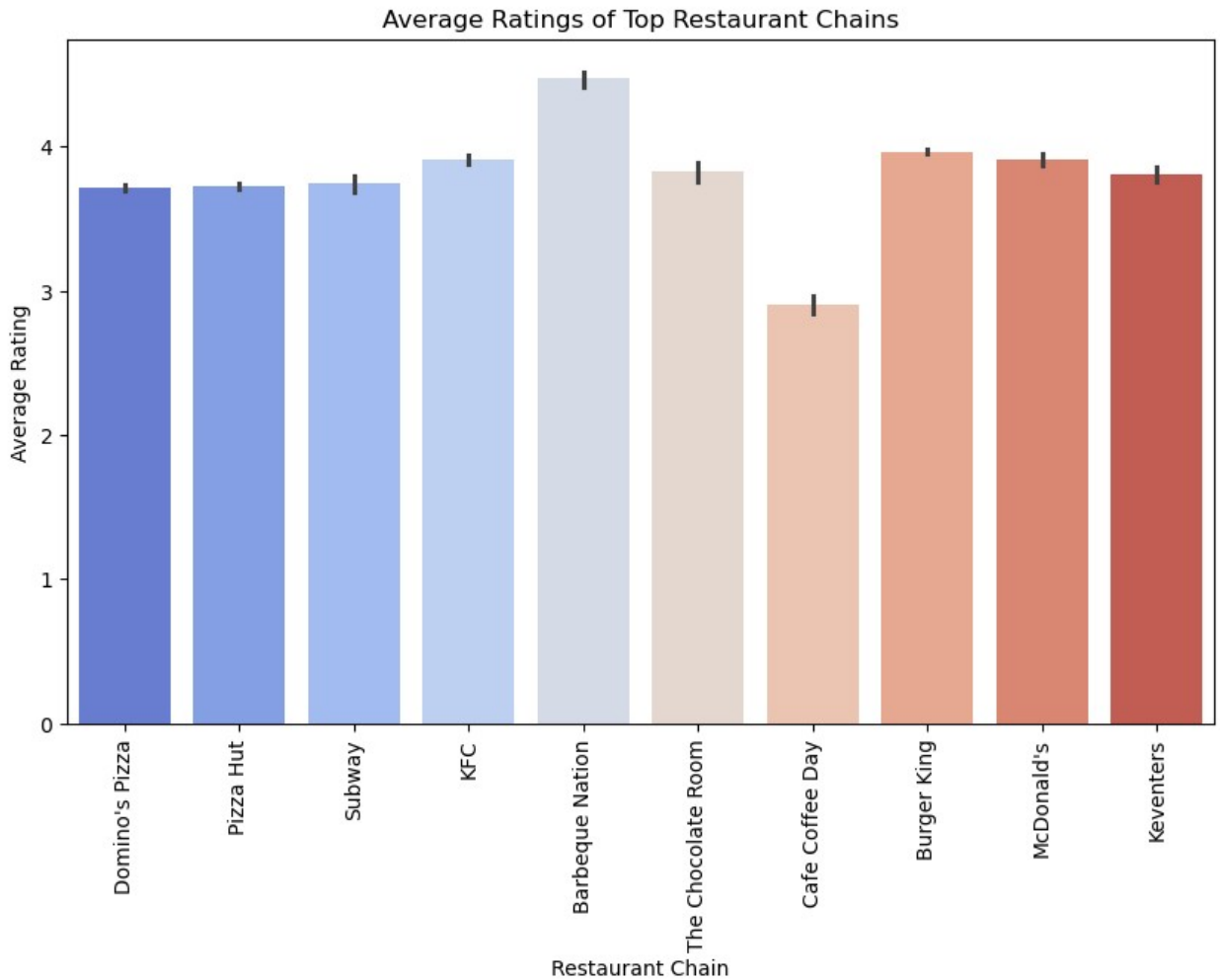

```
plt.pie(chain_counts.values, labels=chain_counts.index, autopct='%1.1f
%%')
centre_circle = plt.Circle((0,0),0.70,fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title("Top Restaurant Chains (Donut Chart)")
plt.show()
```



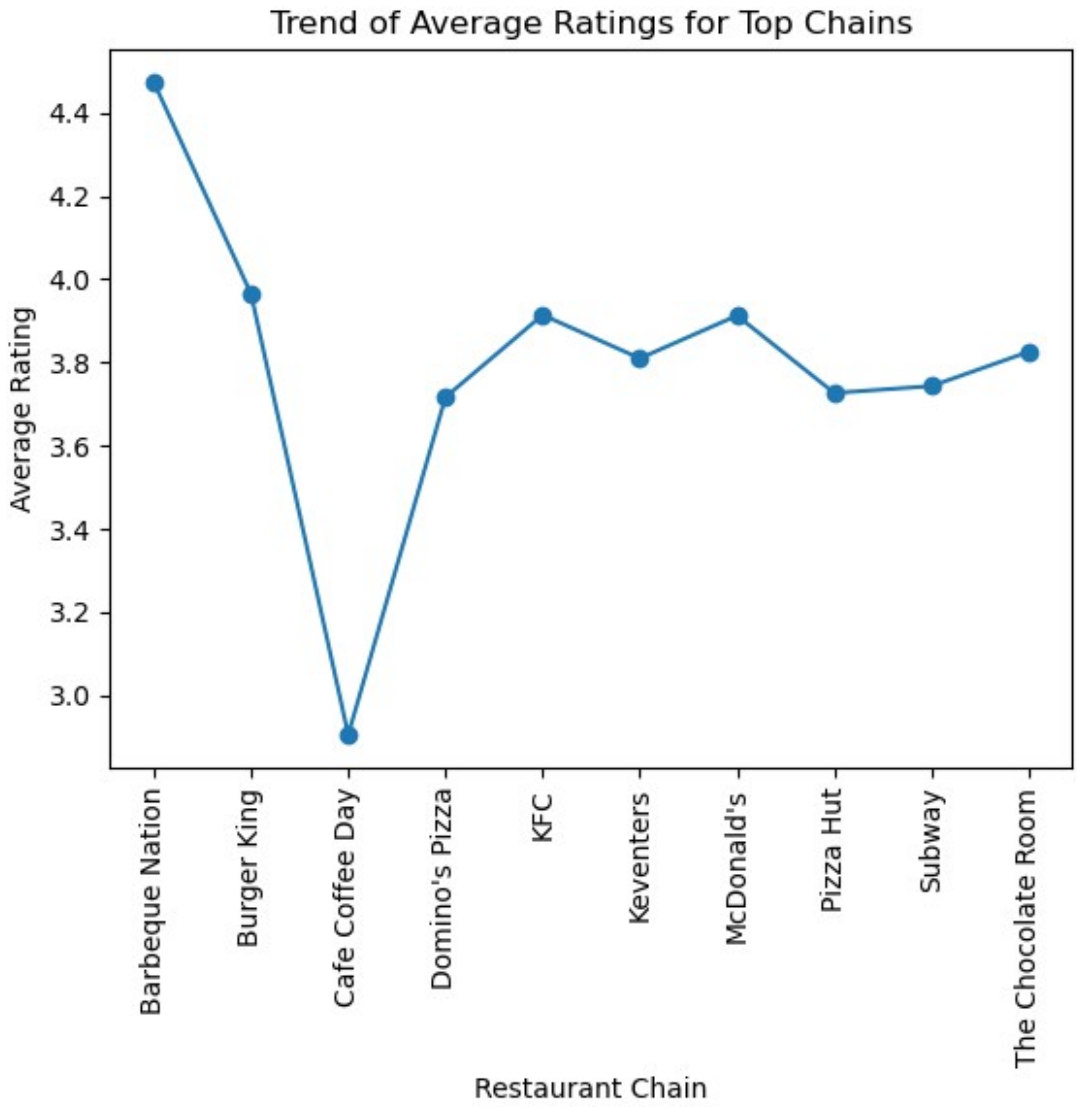

```
#Get Top 10 Restaurant Chain
top_chains = df['name'].value_counts().head(10).index
#Filter only those restaurants
top_chain_data = df[df['name'].isin(top_chains)]
#Visualization
plt.figure(figsize=(10,6))
sns.boxplot(x='name', y='aggregate_rating', data=top_chain_data)
plt.xticks(rotation=90)
plt.title("Rating Distribution of Top Restaurant Chains (Boxplot)")
plt.xlabel("Restaurant Chain")
plt.ylabel("Rating")
plt.show()
```




```
plt.figure(figsize=(10,6))
sns.barplot(x='name', y='aggregate_rating', data=top_chain_data,b
estimator='mean', palette='coolwarm')
plt.xticks(rotation=90)
plt.title("Average Ratings of Top Restaurant Chains")
plt.xlabel("Restaurant Chain")
plt.ylabel("Average Rating")
plt.show()
```




```
avg_chain_rating = top_chain_data.groupby('name')
['aggregate_rating'].mean()
plt.plot(avg_chain_rating.index, avg_chain_rating.values, marker='o')
plt.xticks(rotation=90)
plt.title("Trend of Average Ratings for Top Chains")
plt.xlabel("Restaurant Chain")
plt.ylabel("Average Rating")
plt.show()
```



Restaurant Features:

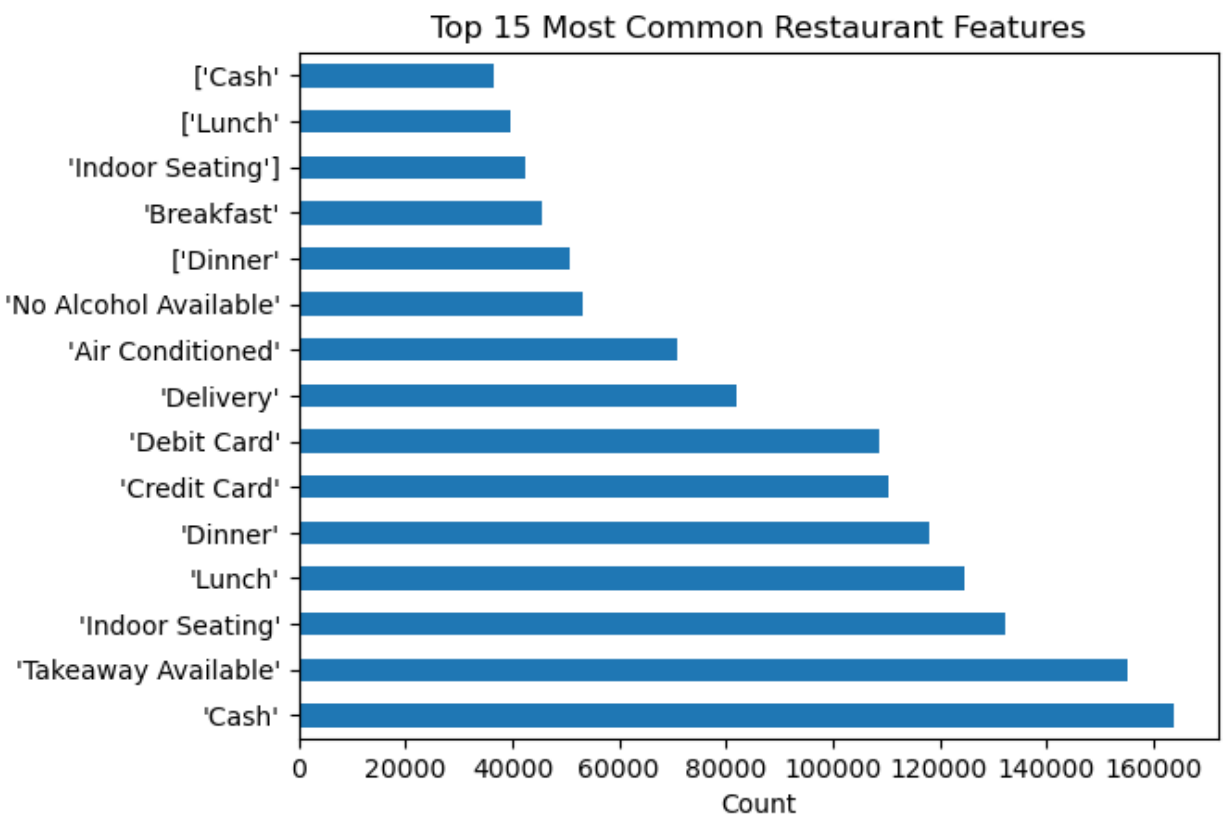
Analyze the distribution of restaurants based on features like Wi-Fi, Alcohol availability, etc.

Investigate if the presence of certain features correlates with higher ratings.

```
df['features'] = df['highlights'].str.split(',')
feature_list = []
for row in df['features']:
    for f in row:
        feature_list.append(f.strip())
feature_counts = pd.Series(feature_list).value_counts().head(15)
print(feature_counts)
```

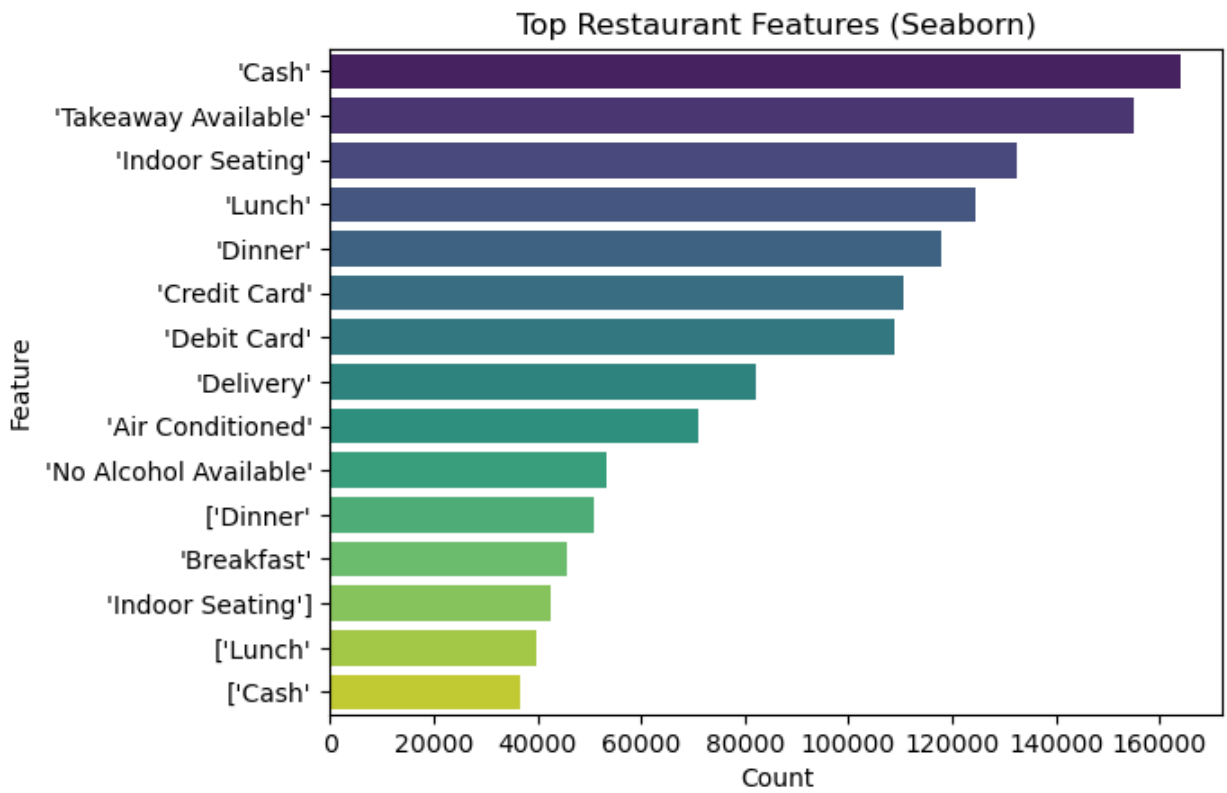
```
feature_counts.plot(kind='barh')
plt.title("Top 15 Most Common Restaurant Features")
plt.xlabel("Count")
plt.show()
```

```
'Cash'                      163988
'Takeaway Available'        155067
'Indoor Seating'            132420
'Lunch'                     124649
'Dinner'                    117927
'Credit Card'               110577
'Debit Card'                108802
'Delivery'                   82039
'Air Conditioned'            70888
'No Alcohol Available'       53196
['Dinner'                    50775
'Breakfast'                  45526
'Indoor Seating']            42554
['Lunch'                     39744
['Cash'                      36547
Name: count, dtype: int64
```

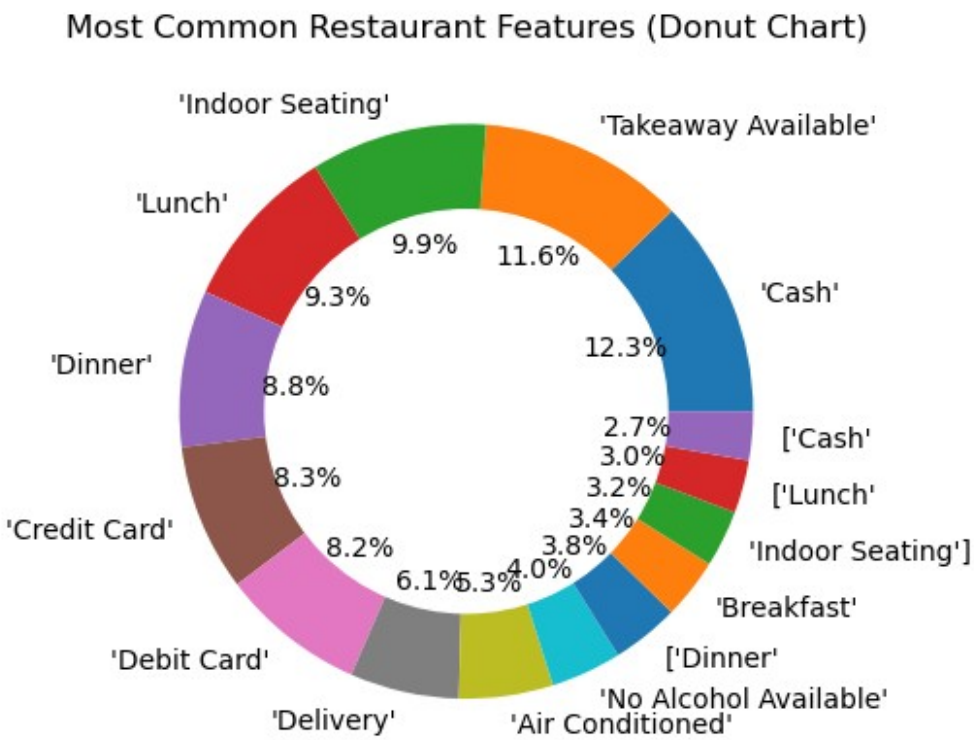



```
sns.barplot(x=feature_counts.values, y=feature_counts.index,
palette='viridis')
```

```
plt.title("Top Restaurant Features (Seaborn)")
plt.xlabel("Count")
plt.ylabel("Feature")
plt.show()
```

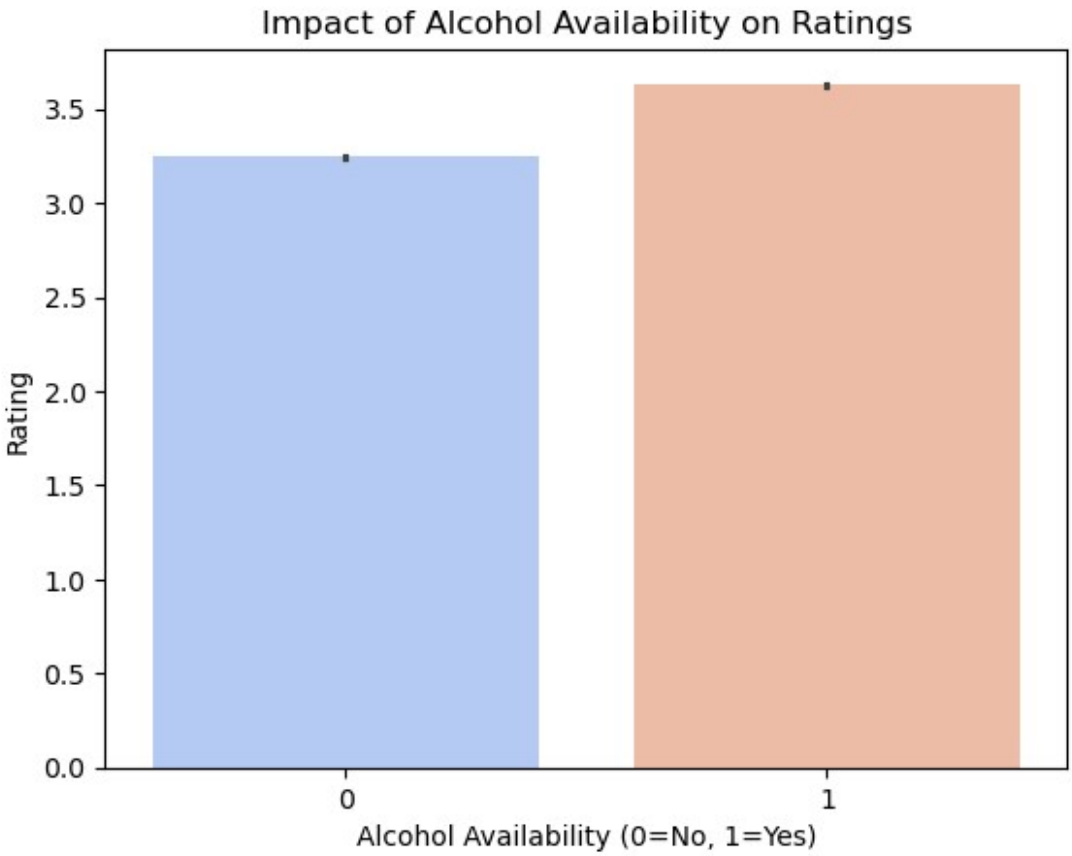



```
plt.pie(feature_counts.values, labels=feature_counts.index,
autopct="%1.1f%%")
circle = plt.Circle((0,0),0.7,color='white')
plt.gca().add_artist(circle)
plt.title("Most Common Restaurant Features (Donut Chart)")
plt.show()
```

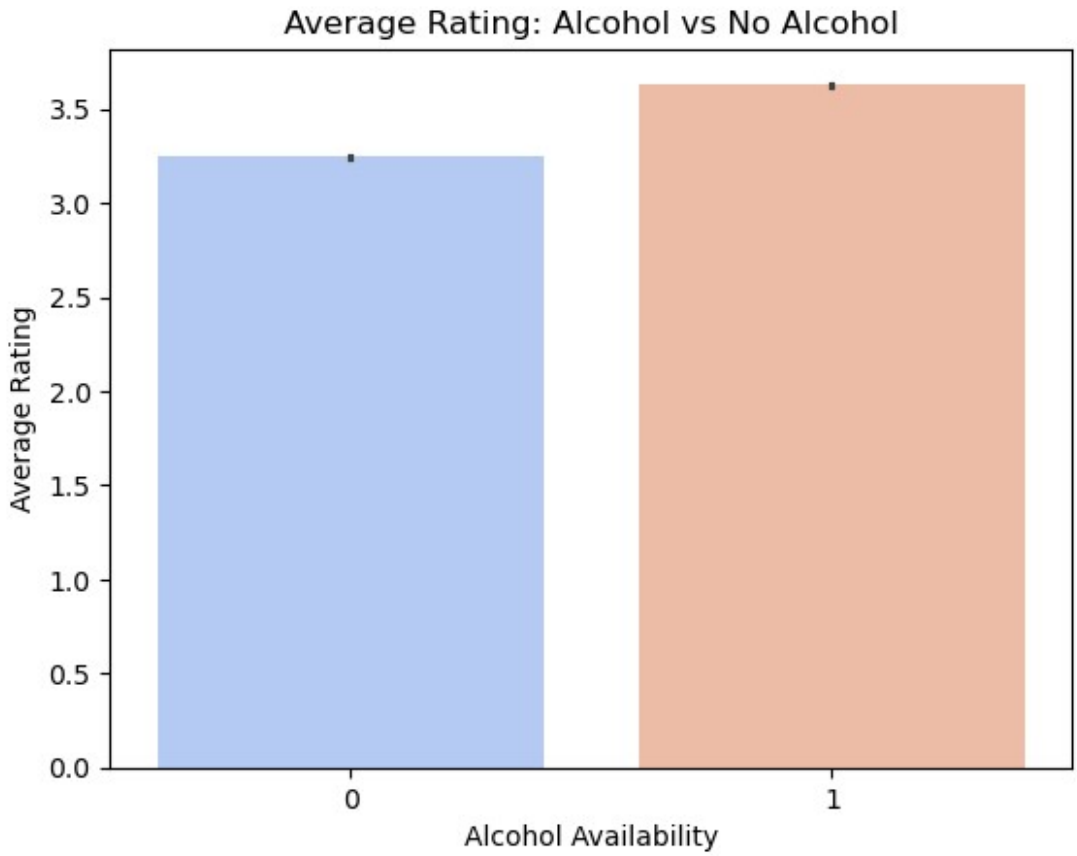


Analyze the distribution of restaurants that offer table booking.

```
df['has_alcohol'] = df['highlights'].str.contains('Alcohol',
case=False).astype(int)
df['has_wifi'] = df['highlights'].str.contains('Wifi',
case=False).astype(int)
df['has_outdoor'] = df['highlights'].str.contains('Outdoor',
case=False).astype(int)
df.groupby("has_alcohol")["aggregate_rating"].mean()
sns.barplot(x='has_alcohol', y='aggregate_rating',palette =
'coolwarm', data=df)
plt.xlabel("Alcohol Availability (0=No, 1=Yes)")
plt.ylabel("Rating")
plt.title("Impact of Alcohol Availability on Ratings")
plt.show()
```

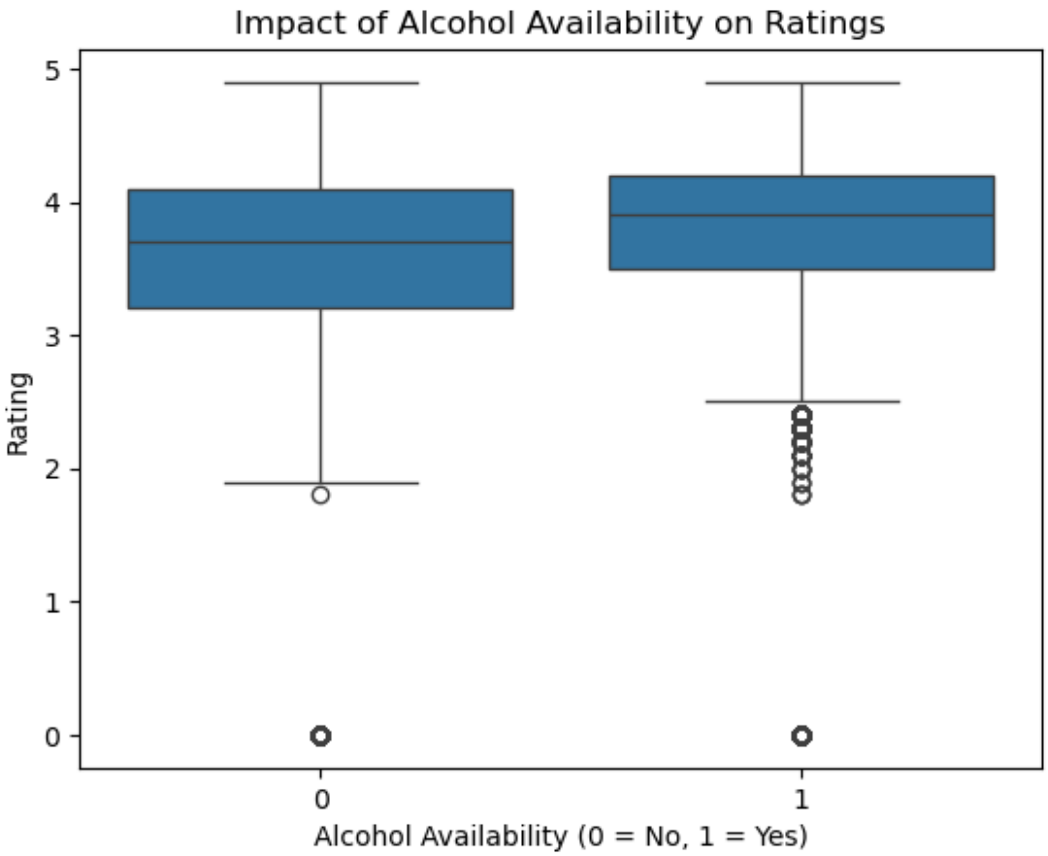



```
sns.barplot(x='has_alcohol', y='aggregate_rating', data=df,
palette='coolwarm')
plt.title("Average Rating: Alcohol vs No Alcohol")
plt.xlabel("Alcohol Availability")
plt.ylabel("Average Rating")
plt.show()
```

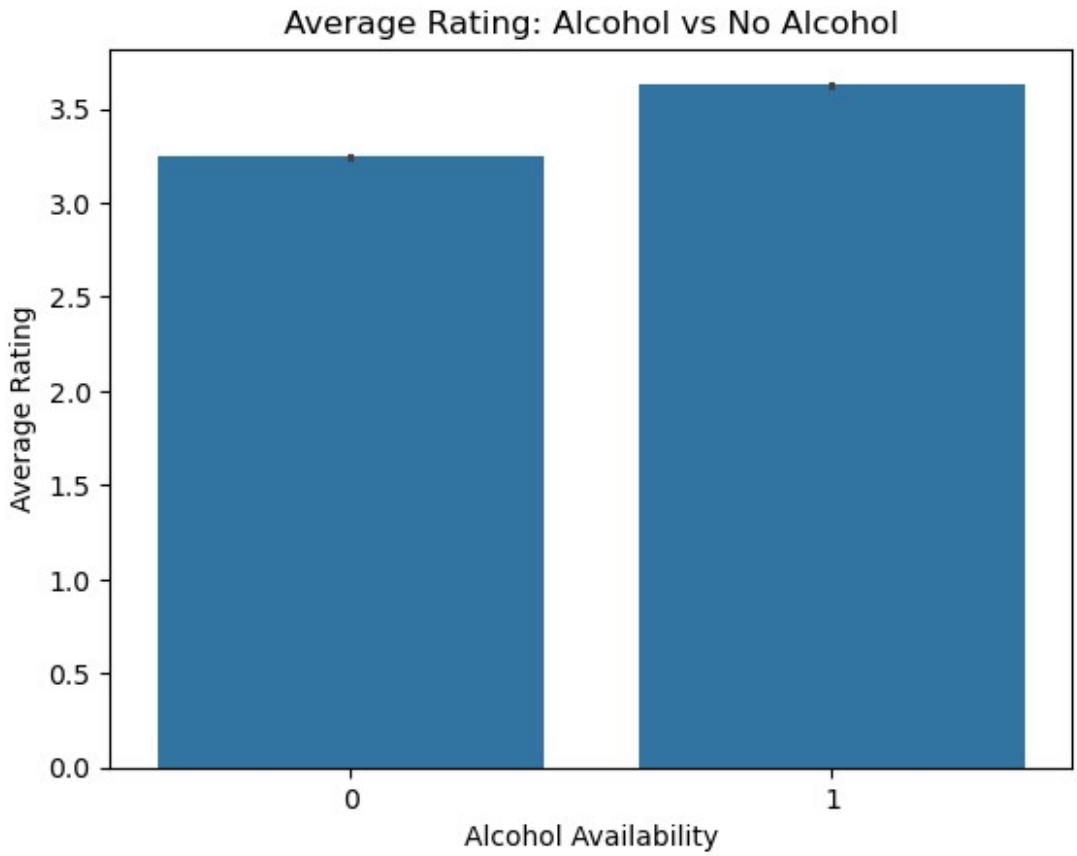


Investigate if the presence of certain features correlates with higher ratings

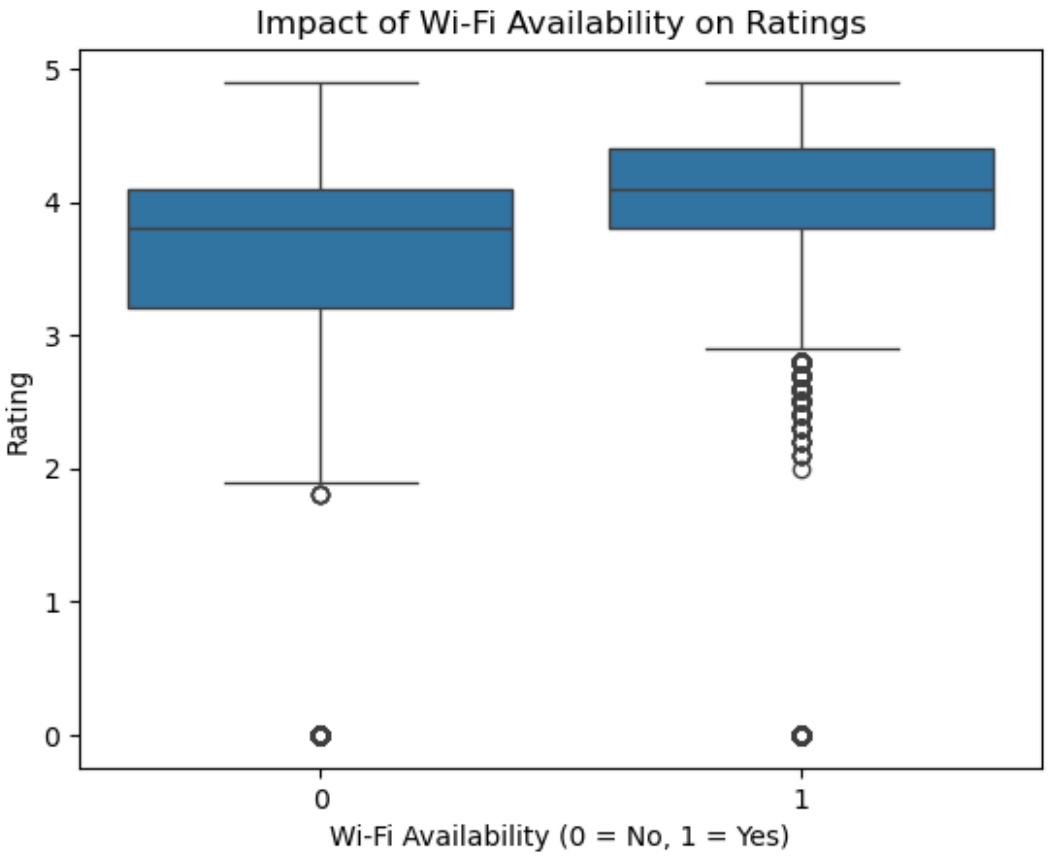
```
df['aggregate_rating'] = pd.to_numeric(df['aggregate_rating'],
errors='coerce')
sns.boxplot(x='has_alcohol', y='aggregate_rating', data=df)
plt.xlabel("Alcohol Availability (0 = No, 1 = Yes)")
plt.ylabel("Rating")
plt.title("Impact of Alcohol Availability on Ratings")
plt.show()
```



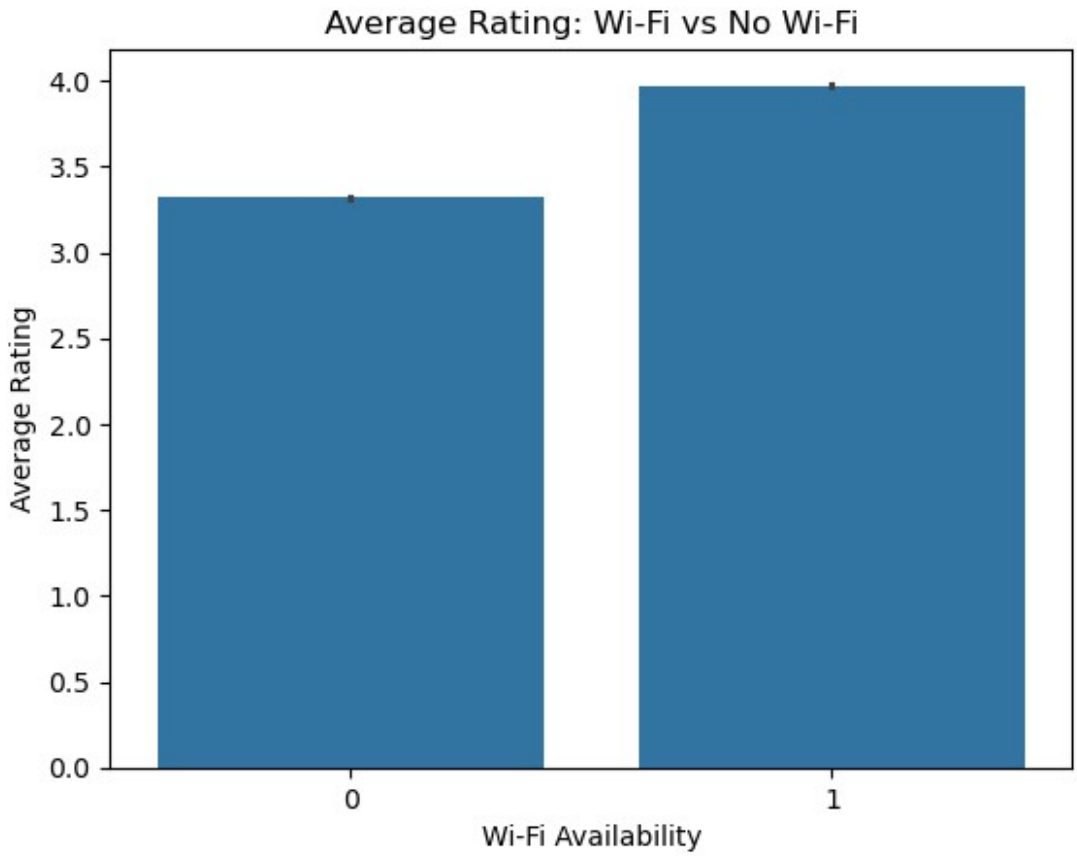

```
sns.barplot(x='has_alcohol', y='aggregate_rating', data=df)
plt.xlabel("Alcohol Availability")
plt.ylabel("Average Rating")
plt.title("Average Rating: Alcohol vs No Alcohol")
plt.show()
```



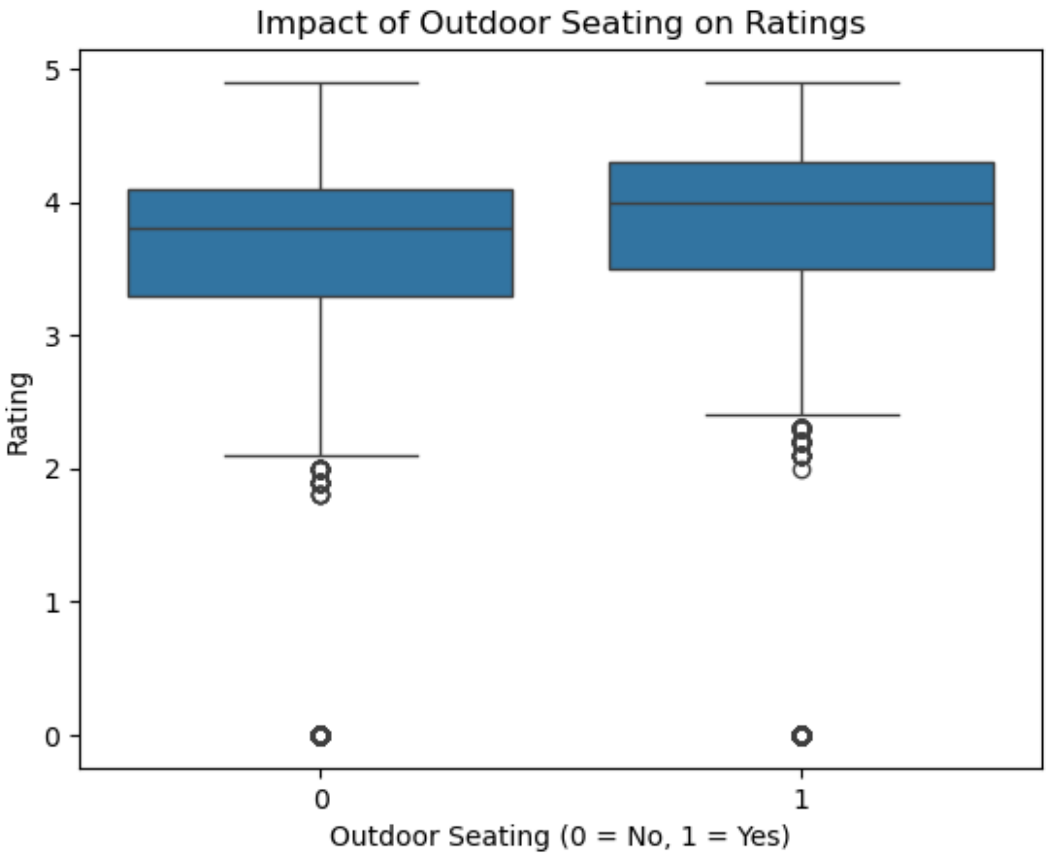

```
sns.boxplot(x='has_wifi', y='aggregate_rating', data=df)
plt.xlabel("Wi-Fi Availability (0 = No, 1 = Yes)")
plt.ylabel("Rating")
plt.title("Impact of Wi-Fi Availability on Ratings")
plt.show()
```



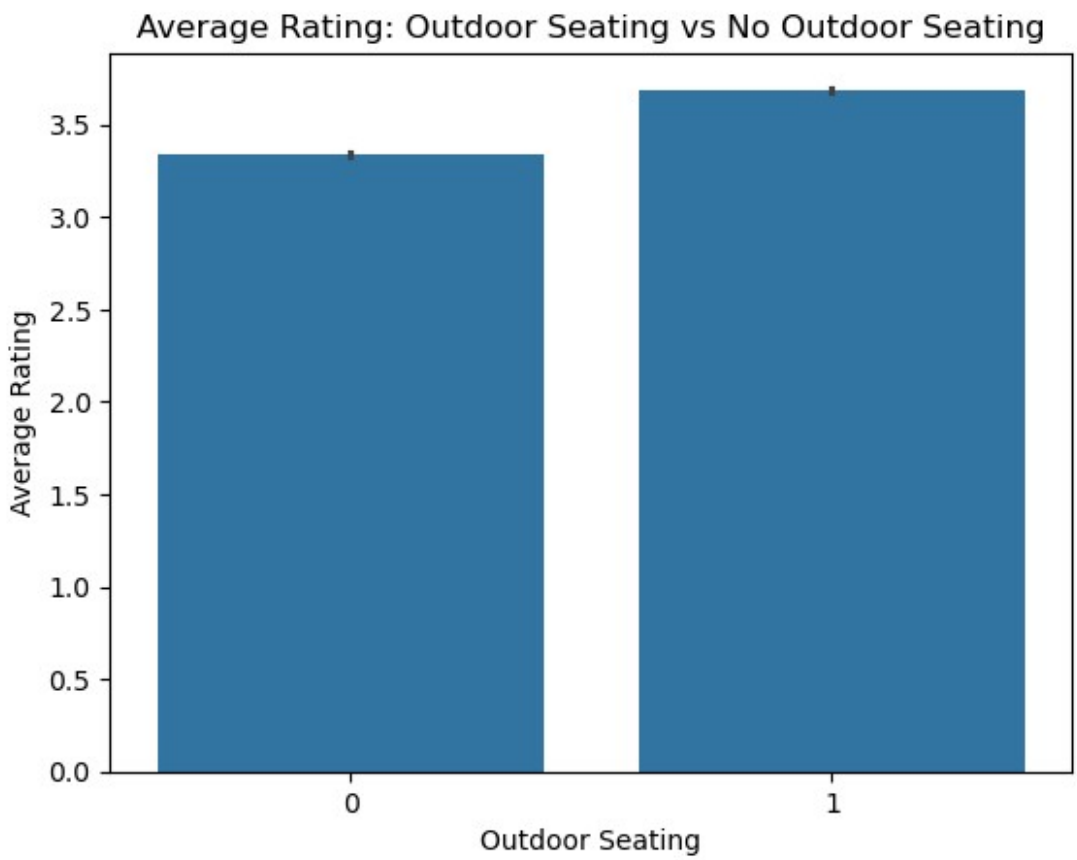

```
sns.barplot(x='has_wifi', y='aggregate_rating', data=df)
plt.xlabel("Wi-Fi Availability")
plt.ylabel("Average Rating")
plt.title("Average Rating: Wi-Fi vs No Wi-Fi")
plt.show()
```




```
sns.boxplot(x='has_outdoor', y='aggregate_rating', data=df)
plt.xlabel("Outdoor Seating (0 = No, 1 = Yes)")
plt.ylabel("Rating")
plt.title("Impact of Outdoor Seating on Ratings")
plt.show()
```




```
sns.barplot(x='has_outdoor', y='aggregate_rating', data=df)
plt.xlabel("Outdoor Seating")
plt.ylabel("Average Rating")
plt.title("Average Rating: Outdoor Seating vs No Outdoor Seating")
plt.show()
```

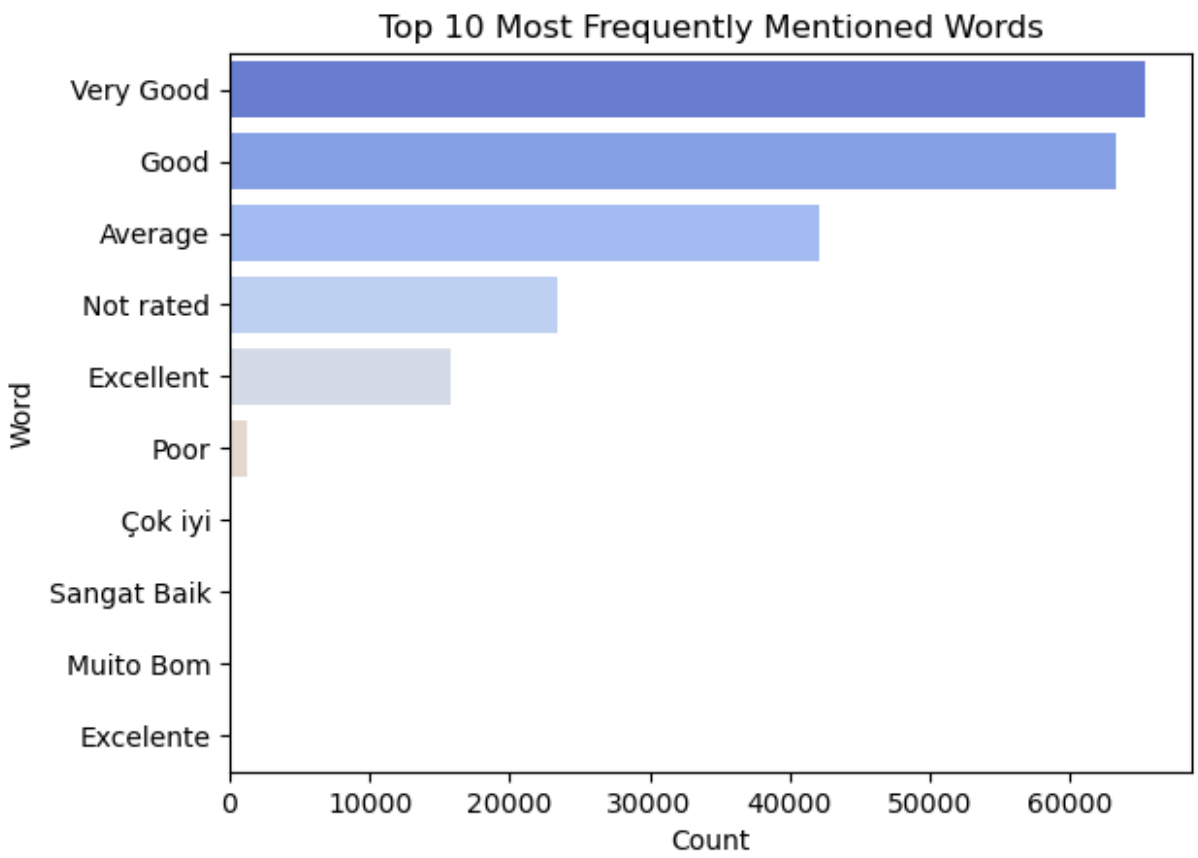


Analyze frequently mentioned words and sentiments.

```
from collections import Counter

df['rating_text'] = df['rating_text'].fillna('')

word_counts = Counter(df['rating_text'])

print(word_counts)
print('--------------------------------MOST
COMMON---------------------------------')
print(word_counts.most_common(10))

positive = ['Excellent', 'Very Good', 'Good']
positive_counts = {word: word_counts[word] for word in positive}
print('--------------------------------Positive Words and Their
Counts---------------------------------')
print("Positive Sentiment Counts:")
positive = ['Excellent', 'Very Good', 'Good']
positive_counts = {word: word_counts[word] for word in positive}

print("Positive Sentiment Counts:")
print(positive_counts)
```

```
print('-------------------------------Negative Words and Their
Counts----------------------------------')
negative = ['Poor', 'Average']
negative_counts = {word: word_counts[word] for word in negative}

print("Negative Sentiment Counts:")
print(negative_counts)
print('-------------------------------Seaborn Plot for Top
Words----------------------------------------')
top_words = pd.DataFrame(word_counts.most_common(10),
columns=['word','count'])

sns.barplot(x='count', y='word',palette = 'coolwarm', data=top_words)
plt.xlabel("Count")
plt.ylabel("Word")
plt.title("Top 10 Most Frequently Mentioned Words")
plt.show()
```

```
Counter({'Very Good': 65451, 'Good': 63384, 'Average': 42157, 'Not
rated': 23478, 'Excellent': 15737, 'Poor': 1175, 'Çok iyi': 56,
'Sangat Baik': 44, 'Muito Bom': 44, 'Excelente': 42, 'Muy Bueno': 35,
'Bardzo dobrze': 31, 'Bom': 26, 'Skvělé': 25, 'Baik': 24, 'Velmi
dobré': 22, 'Harika': 22, 'İyi': 19, 'Ottimo': 18, 'Veľmi dobré': 17,
'Terbaik': 16, 'Buono': 14, 'Skvělá volba': 13, 'Dobré': 12, 'Bueno':
11, 'Dobrze': 9, 'Wybitnie': 8, 'Eccellente': 8, 'Vynikajúce': 7,
'Průměr': 6, 'Muito bom': 6, 'Média': 5, 'Promedio': 5, 'Ortalama': 3,
'Scarso': 3, 'Średnio': 3, 'Priemer': 3, 'Media': 3, 'Biasa': 2})
-------------------------------MOST
COMMON----------------------------------
[('Very Good', 65451), ('Good', 63384), ('Average', 42157), ('Not
rated', 23478), ('Excellent', 15737), ('Poor', 1175), ('Çok iyi', 56),
('Sangat Baik', 44), ('Muito Bom', 44), ('Excelente', 42)]
-------------------------------Positive Words and Their
Counts----------------------------------
Positive Sentiment Counts:
Positive Sentiment Counts:
{'Excellent': 15737, 'Very Good': 65451, 'Good': 63384}
-------------------------------Negative Words and Their
Counts----------------------------------
Negative Sentiment Counts:
{'Poor': 1175, 'Average': 42157}
-------------------------------Seaborn Plot for Top
Words----------------------------------------
```



Key Findings

Ratings Overview

- Average rating of restaurants is around 3.4, indicating overall positive customer satisfaction.
- Most ratings lie between 3.0 and 4.5.

Location Insights

- Some cities have significantly more restaurants than others, showing higher food market density.
- Average ratings also vary across cities, helping identify cities with better-rated restaurants.

Popular Cuisines

- The most popular cuisines include North Indian, Chinese, Fast Food, and South Indian.
- Multi-cuisine restaurants are common.

Cuisine Variety vs Rating

- Restaurants with 2–3 cuisines tend to have slightly better ratings.
- Variety has a weak positive impact on ratings.

Price Range Insights

- Higher price ranges have higher average costs (expected).
- There is no strong linear relationship between price range and rating—good food exists in all budget categories.

Online Ordering Impact

- Restaurants offering online delivery tend to show slightly higher ratings.
- This indicates customer preference for convenience.

Table Booking (OpenTable Support)

- Only a small portion of restaurants support table booking.
- Ratings are slightly higher for restaurants with table booking.

Top Restaurant Chains

- Some chains dominate the market with many outlets.
- Their ratings show different levels of consistency.

Restaurant Features

- Features like Wi-Fi, Alcohol Availability, and Outdoor Seating show a positive relationship with ratings.
- Restaurants offering Alcohol and Outdoor Seating generally receive higher average ratings.

Insights

Customer Preferences

- Customers respond positively to restaurants with good ambience, multiple cuisines, and additional features (Wi-Fi, Alcohol, Outdoor Seating).
- Online delivery is a major factor influencing ratings.

Market Trends

- Certain cities are clear food hubs with high restaurant density.
- Popular cuisines dominate the market and influence competition.

Business Recommendations

- Restaurants can consider adding features such as Wi-Fi or Outdoor Seating to increase customer satisfaction.
- Offering multiple cuisines can attract wider audiences.
- Enabling online ordering boosts convenience and enhances ratings.
- Monitoring customer reviews and sentiment can help address quality issues.

Conclusion

The exploratory data analysis of the Indian Restaurants dataset provided useful insights into restaurant characteristics, customer preferences, and factors affecting ratings. Overall, the restaurants in the dataset show generally positive customer sentiment, with most ratings falling between 3.0 and 4.5. Cuisine variety, cost for two, availability of online ordering, restaurant chains, and special features such as Wi-Fi or alcohol availability all show measurable relationships with customer satisfaction. Although seasonal trends could not be directly analyzed due to a lack of date information, synthetic monthly analysis helped illustrate how such trends could be studied in a real-world dataset.

The analysis reveals strong patterns in city-wise distribution, cuisine popularity, and feature-based differences in ratings. These findings can help restaurant owners and platforms like Zomato understand what customers value the most and how certain features contribute to higher ratings.